

# Dotter: Mathematical Analysis and Implementation Guide

Oscar Laird

March 2026

# Contents

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Table of Contents</b>                                                     | <b>1</b>  |
| <b>Introduction</b>                                                          | <b>3</b>  |
| <b>1 Architecture</b>                                                        | <b>4</b>  |
| 1.1 The Likelihood Cycle . . . . .                                           | 5         |
| 1.2 The Prior Cycle . . . . .                                                | 5         |
| 1.3 Client-Server Communication . . . . .                                    | 5         |
| <b>2 The Bayesian Trie</b>                                                   | <b>6</b>  |
| 2.1 Lazy Posterior Calculation: The Core Algorithms . . . . .                | 6         |
| 2.2 Likelihood Tracking . . . . .                                            | 13        |
| 2.3 Character Priors from Token Models . . . . .                             | 14        |
| 2.4 Prior Tracking . . . . .                                                 | 19        |
| 2.5 Maximum Descendant Likelihood . . . . .                                  | 23        |
| 2.6 The Complete Algorithm . . . . .                                         | 28        |
| 2.7 OLD STUFF . . . . .                                                      | 29        |
| <b>3 Determine Visible Nodes</b>                                             | <b>31</b> |
| 3.1 Setting the Visibility Threshold . . . . .                               | 31        |
| 3.2 Searching for Visible Nodes . . . . .                                    | 31        |
| 3.3 Responding to Likelihood Updates versus Prior Updates . . . . .          | 31        |
| <b>4 Render Trie</b>                                                         | <b>33</b> |
| 4.1 Embellishments . . . . .                                                 | 33        |
| 4.1.1 Branches of the Tree . . . . .                                         | 33        |
| <b>5 Stimulus Optimization: Ideal Timer Placement</b>                        | <b>34</b> |
| 5.1 The Continuous One Bit Channel . . . . .                                 | 34        |
| 5.1.1 Two Interpretations . . . . .                                          | 34        |
| 5.1.2 The Ideal Timing Distribution . . . . .                                | 34        |
| 5.2 Periodic Signals . . . . .                                               | 35        |
| 5.2.1 Random Scan Times . . . . .                                            | 35        |
| 5.2.2 Periodic signals . . . . .                                             | 35        |
| 5.2.3 Approximating A Uniform Distribution with a Gaussian Mixture . . . . . | 35        |
| 5.3 User Parameters . . . . .                                                | 38        |
| 5.3.1 The Likelihood Model . . . . .                                         | 38        |
| 5.3.2 Directional Statistics . . . . .                                       | 39        |
| <b>6 Render Stimulus</b>                                                     | <b>40</b> |
| 6.1 Circular Timers . . . . .                                                | 40        |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| <b>7</b>  | <b>Detect Response</b>                                          | <b>41</b> |
| 7.1       | Blink Detection . . . . .                                       | 41        |
| 7.2       | Keypress Detection . . . . .                                    | 41        |
| <b>8</b>  | <b>Determine Most Likely Break Nodes</b>                        | <b>42</b> |
| 8.1       | An Upper Bound for the Likelihood of a Token Sequence . . . . . | 42        |
| 8.2       | Descending the Token Trie . . . . .                             | 43        |
| <b>9</b>  | <b>LLM Query</b>                                                | <b>44</b> |
| 9.1       | Maintaining a Key-Value Cache Trie . . . . .                    | 44        |
| <b>10</b> | <b>Mastering Dotter</b>                                         | <b>45</b> |
| 10.1      | The Cost of Incorrect Signals . . . . .                         | 45        |
|           | <b>Bibliography</b>                                             | <b>46</b> |

# Introduction

Dotter is a text-entry technology that allows users to type over a 1-bit channel. It allows a simple input mechanism such as blinking or pressing a single button to be used to type. Dotter milks as much information out of that 1-bit channel as possible, allowing an experienced user to type standard english text at speeds of 20 words per minute only by blinking.

**Dotter only has one rule:** Identify the longest visible prefix of your target text and synchronize your signal to its timer.

Dotter was inspired by the project “Dasher” [2], a text-entry technology developed by David MacKay’s group at the University of Cambridge that took inspiration from arithmetic coding to map the space of all possible text sequences to the unit interval. Assuming a version of Fitt’s law, Dasher is an optimal text-entry technology for 1-dimensional continuous (pointing) input.

This document is a mathematical analysis of Dotter as well as an implementation guide, describing the architecture and key algorithms of Dotter. Chapter 1 describes the architecture of Dotter, outlining its key data structures and algorithms. The succeeding chapters describe individual components of Dotter. The final chapter gives theory and advice on how to learn to type with Dotter. (If the person reading this only wants to learn how to use Dotter, they should skip to the final chapter.)

The name “Dotter” is a nod the most famous 1-bit text-entry system, telegraphy, which consisted of “dots” and “dashes.” The name “Dasher” was already taken, so I went with this.

# Chapter 1

## Architecture

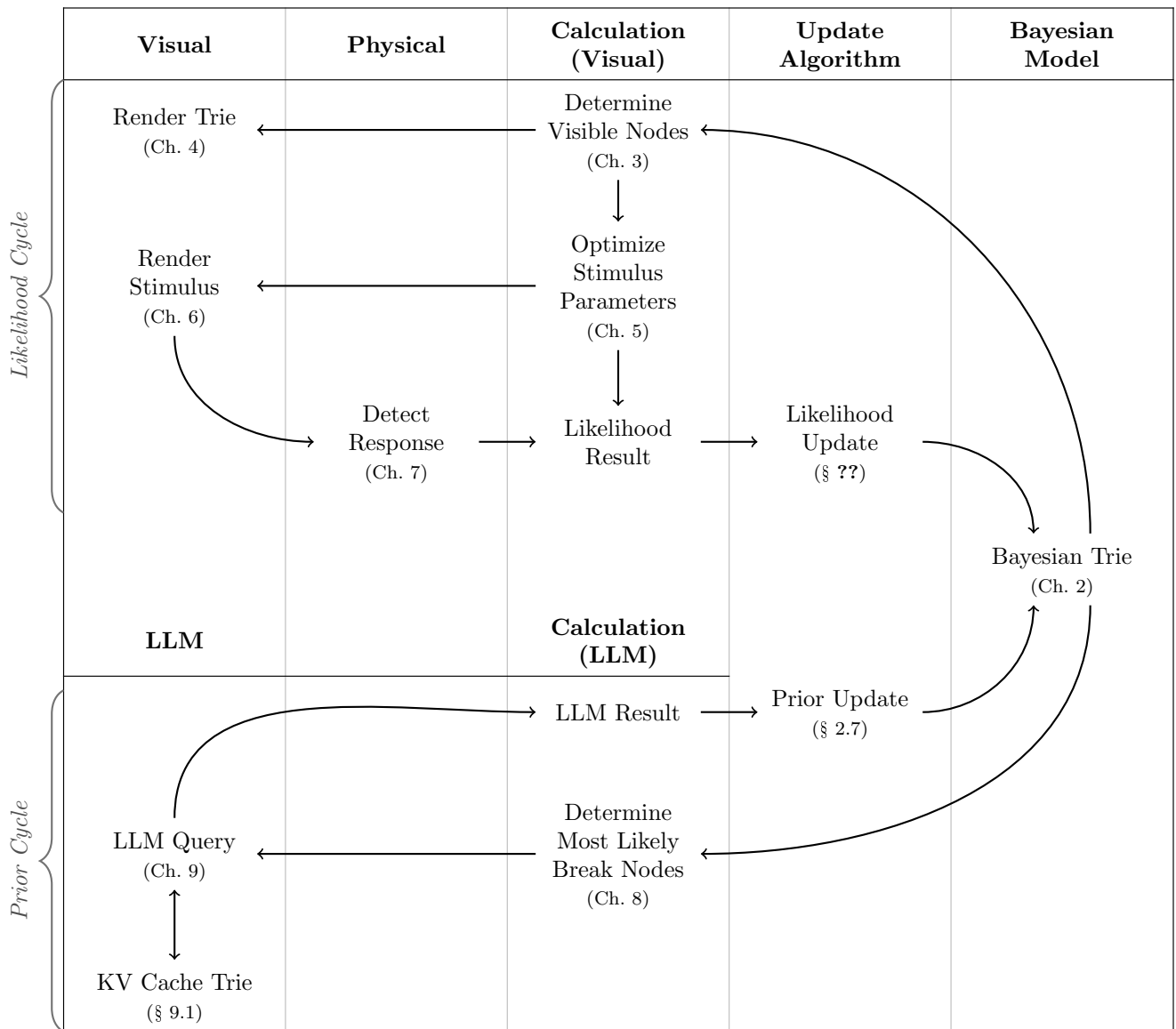


Table 1.1: Architecture of Dotter

Table 1.1 shows the architecture of Dotter. At its heart is the “bayesian trie” (chapter 2) in the rightmost column, which is a character-based Bayesian prefix trie which maintains a calculation of the posterior probability of every prefix. The other elements of the system may be separated into the Likelihood Cycle and the Prior Cycle.

Each element in Table 1.1 is discussed in detail in its own chapter or section.

## 1.1 The Likelihood Cycle

The Likelihood Cycle refers to the series of steps by which we elicit a signal from the user and then interpret that signal to update our posterior probabilities. It has seven steps,

1. **Determine Visible Nodes** (chapter 3): we determine which prefixes should be visually displayed by checking which prefixes have a posterior probability exceeding the visibility threshold (e.g. 2%).
2. **Render Trie** (chapter 4): here we draw the visible prefixes to the screen, connecting parent nodes to their children with smooth curves.
3. **Optimize Stimulus Parameters** (chapter 5): in this step we assign timers to every prefix. We want to maximize the information gain from the user’s signal which is accomplished by making the timers as far apart from each other as possible (in a formal sense). This helps to make the user’s selection unambiguous.
4. **Render Stimulus** (chapter 6): we draw the timers to the screen.
5. **Detect Response** (chapter 7): we detect the user’s signal. It is trivial to detect a keypress; detecting the exact timing of a blink is a more sophisticated problem.
6. **Likelihood Result**: we compare the detected user signal to the timers that were assigned in step 3. This gives us a likelihood score for each prefix.
7. **Likelihood Update** (§ ??): we update the trie to reflect the received likelihoods. Performing this update in a lazy and efficient manner is a central technical challenge of Dotter. (This is an “update” in the traditional bayesian sense.)

## 1.2 The Prior Cycle

The Prior Cycle refers to the series of steps by which we query a large language model to refine our prior probabilities and incorporate those results into our posterior beliefs. It has four steps,

1. **Determine Most Likely Break Nodes** (chapter 8): we determine which nodes are most likely to a break between tokens in the tokenization of the user’s target text.
2. **LLM Query** (chapter 9): we query the large language model to predict the distribution over the next token for the prefixes identified in the previous step, making use of the Key-Value Cache Trie (§ 9.1).
3. **LLM Result**: we receive the result from the language model.
4. **Prior Update** (§ 2.7): we update the trie to reflect the received prior probabilities. Performing this update in a lazy and efficient manner is a central technical challenge of Dotter. (This is not truly an “update” in the traditional bayesian sense, since normally our priors are fixed.)

## 1.3 Client-Server Communication

A web-based client is responsible for the likelihood cycle, while a server with a large language model handles the prior cycle. Since these two cycles must run independently, it is necessary for both the client and the server to maintain a copy of the trie.

The data designated “Likelihood Result” and “LLM Result” are exchanged between the client and the server, with each side updating its copy of the trie in response to the stream of these results.

The two sides will not necessarily receive this stream of results in the same order, but an important correctness property of our update algorithms is that the state of the trie does not depend on the order in which the updates are performed.

## Chapter 2

# The Bayesian Trie

Dotter must determine the most likely string that the user is typing, based on observation of the user's signals. To accomplish this, Dotter maintains a trie indicating the posterior probability that any given prefix is a prefix of the user's target string.

To compare strings, we use the inequality notation  $X \leq Y$  to mean that the string  $X$  is a prefix of the string  $Y$ .

Letting  $X$  be the user's target string,  $D$  be the user's signals, then the posterior probability that a given string,  $S$ , is a prefix of  $X$  is given by:

$$P(S \leq X|D) = \frac{P(D|S \leq X)P(S \leq X)}{P(D)}$$

In practice, we only keep track of the unnormalized posterior probabilities, denoted in code as `post_Z` (to mean the posterior probability multiplied by a normalizing constant). Since the root node,  $S_0$ , is the empty string, we have  $P(S_0 \leq X) = 1$  and thus the unnormalized posterior probability of the root node is  $P(D)$ , i.e., the normalizing constant. Thus if we know the unnormalized posterior probabilities in our trie, we can convert these to normalized posterior probabilities by dividing by the root node's unnormalized posterior probability.

The posterior probabilities are derived from the prior probabilities and the likelihoods. In Dotter, both are subject to updates. The likelihood is updated as we receive signals from the user and the prior is updated as our language model refines our predictions.

Our analysis has five parts:

1. We assume a character-based prior model, present algorithms for lazy calculation of the unnormalized posteriors, and prove their correctness.
2. We discuss implementation details with regard to tracking the likelihoods.
3. We discuss converting a token-based prior model to a character-based prior model.
4. We discuss how to calculate the branch prior for a node.
5. We discuss how to track the maximum descendant likelihoods of token sequences.
6. We present the full algorithms.

## 2.1 Lazy Posterior Calculation: The Core Algorithms

**Definition 2.1** (Alphabet). *An alphabet is a finite set.*

**Definition 2.2** (Character). *A character is an element of an alphabet.*

**Definition 2.3** (String). *A string is a sequence of characters.*

**Definition 2.4** (Substring). *We use the notation  $s[i : j]$  to represent the subsequence of characters of  $s$  from index  $i$  to index  $j - 1$ . We also allow the notation  $s[: -k]$  to represent the subsequence of characters of  $s$  from index 0 to index  $|s| - k - 1$ .*

**Definition 2.5** (String Inequality). *We say that a string  $s$  is a prefix of a string  $t$ , and write this as  $s \leq t$ , if and only if there exists an integer  $k$  such that  $s = t[0 : k]$ .*

**Definition 2.6** (Properly Terminated String). *A string  $h$  is properly terminated if and only if  $h[i] = \blacksquare' \iff i = |h| - 1$ . An unterminated string is one that does not contain the stop character. An “at most properly terminated string” is one that is either properly terminated or unterminated.*

Throughout this chapter, we will let  $\mathcal{A}$  denote our alphabet which must include at least the two special characters: space (' ') and stop (' $\blacksquare$ '); the set of all strings over this alphabet will be denoted  $S$ , and the subset of all properly terminated strings will be denoted  $H$ , where if  $h \in H$ .

**Definition 2.7** (Prefix Code). *A prefix code is a set of strings such that no string is a prefix of another string in the set, i.e., A set  $C$  is a prefix code if and only if,*

$$\nexists c_1, c_2 \in C, c_1 < c_2$$

**Definition 2.8** (Complete Prefix Code). *A complete prefix code,  $C$ , is a prefix code that covers the entire tree, i.e.,*

$$\forall x \in S, \exists c \in C, x \leq c \vee c \leq x$$

**Definition 2.9** (Refinement). *A prefix code,  $A$ , is a refinement of a prefix code,  $B$ , if every element of  $A$  is an extension of some element of  $B$ , i.e.,*

$$\forall a \in A, \exists b \in B, a \geq b$$

*If this is the case, we write  $A \geq B$ .*

Refinement is a partial order over complete prefix codes. These prefix codes form a lattice, so we may define the meet and join of two prefix codes.

**Definition 2.10** (Meet). *The meet of two prefix codes,  $A$  and  $B$ , is the greatest lower bound of  $A$  and  $B$ , i.e., it is the set of minimal elements of their intersection. We write this as  $A \wedge B$ .*

**Definition 2.11** (Join). *The join of two prefix codes,  $A$  and  $B$ , is the least upper bound of  $A$  and  $B$ , i.e., it is the set of maximal elements of their union. We write this as  $A \vee B$ .*

**Definition 2.12** (Element Refinement). *We overload the comparison operators when the left hand side is a string and the right hand side is a complete prefix code.  $x \leq C$  means that  $x$  is a prefix of some element of  $C$ , whereas  $x > C$  means that  $x$  is a proper extension of some element of  $C$ .*

**Proposition 2.1.** *The set of all properly terminated strings,  $H$ , is a complete prefix code.*

*Proof.* We first show that  $H$  is a prefix code. We proceed by contradiction. Suppose there exist  $h_1, h_2 \in H$  such that  $h_1 < h_2$ . Because  $h_1$  is a strict prefix of  $h_2$ , it must be that  $|h_1| < |h_2|$ . Furthermore, the characters of  $h_2$  must perfectly match  $h_1$  up to index  $|h_1| - 1$ . Thus,

$$h_2[|h_1| - 1] = h_1[|h_1| - 1].$$

Because  $h_1 \in H$ , it is properly terminated, meaning its final character is the stop character:  $h_1[|h_1| - 1] = \blacksquare'$ . Therefore,  $h_2[|h_1| - 1] = \blacksquare'$ . However, because  $h_2 \in H$ , the stop character can only appear at its final index  $|h_2| - 1$ . This implies:

$$|h_1| - 1 = |h_2| - 1 \implies |h_1| = |h_2|,$$

which contradicts the strict prefix requirement  $|h_1| < |h_2|$ . Thus, no such pair  $h_1, h_2$  can exist, and  $H$  is a prefix code.

We now show that  $H$  is a complete prefix code. Let  $x \in S$  be any string. We show this by cases constructively:

- (a) If ' $\blacksquare$ ' is in  $x$ , then let  $i$  be the index of its first occurrence. Then  $x[i + 1] \in H$  and  $x[i + 1] \leq x$ .
- (b) If ' $\blacksquare$ ' is not in  $x$ , then  $x + \blacksquare' \in H$  and  $x < x + \blacksquare'$ .



□

**Definition 2.13** (Likelihood Function). A likelihood function is a mapping from properly terminated strings to non-negative real numbers,  $L_n : H \rightarrow \mathbb{R}^+$ .

**Definition 2.14** (Prior Function). A prior function is a mapping from properly terminated strings to non-negative real numbers,  $P_n : H \rightarrow \mathbb{R}^+$ , satisfying

$$\sum_{h \in H} P_n(h) = 1$$

**Definition 2.15** (Branch Prior Function). We define the branch prior function,  $P_n^{br}(x)$ , of an at most properly terminated string  $x$ , as,

$$P_n^{br}(x) = \sum_{\substack{h \in H \\ h \geq x}} P_n(h)$$

*Terminology Note:* The branch prior function corresponds to our usual notion of a prior over string prefixes, but we choose to name it explicitly as “branch prior” and to restrict our definition of the domain of a “prior function” to properly terminated strings, excluding their prefixes. We make this choice to keep a close correspondence between the definitions of prior and likelihood functions.

*Note:* In our algorithm, we will assume oracular access to the branch prior function.

**Definition 2.16** (Likelihood Sequence). A likelihood sequence is a sequence of likelihood functions,  $\{L_n\}$  where  $\forall h, L_0(h) = 1$ .

**Definition 2.17** (Prior Sequence). A prior sequence is a sequence of prior functions  $\{P_n\}$ .

**Definition 2.18** (Delta Digest). A delta digest is a prefix code and a mapping from that prefix code to non-negative real numbers,  $(C, \Delta)$ . So,

$$\Delta : C \rightarrow \mathbb{R}^+$$

**Definition 2.19** (Prefix Code Truncation). The truncation of a string  $x$  under a prefix code  $C$  refers to the element  $c \in C$  such that  $c \leq x$ . If no such element exists, then the truncation is undefined. We denote the truncation of  $x$  under  $C$  as  $\lfloor x \rfloor_C$ .

**Definition 2.20** (Likelihood Delta Digest Sequence). The likelihood delta digest sequence corresponding to a likelihood sequence  $\{L_n\}$  is a sequence of delta digests  $\{(C_n^L, \Delta_n^L)\}$ , that satisfies:

$$\forall h \in H, L_n(h) = L_{n-1}(h) \Delta_n^L(\lfloor h \rfloor_{C_n^L})$$

**Definition 2.21** (Prior Delta Digest Sequence). This is defined identically, i.e., the prior delta digest sequence corresponding to a prior sequence  $\{P_n\}$  is a sequence of delta digests  $\{(C_n^P, \Delta_n^P)\}$ , that satisfies:

$$\forall h \in H, P_n(h) = P_{n-1}(h) \Delta_n^P(\lfloor h \rfloor_{C_n^P})$$

**Lemma 2.1** (Eventually Constant Likelihoods). Given the cumulative likelihood update frontier,  $\bigvee_{i=1}^N C_i^L$ , and a given node in that frontier,  $f$ , the likelihoods of all properly terminated strings descended from that node are constant with respect to the branch up to time  $N$ , i.e.,

$$\forall n \leq N, \forall h \in H, h \geq f, L_n(h) = \prod_{1 \leq i \leq n} \Delta_i^L(\lfloor f \rfloor_{C_i^L})$$

*Proof.* Recall that  $L_0(h) = 1$ . By definition of the likelihood sequence,

$$L_n(h) = \prod_{1 \leq i \leq n} \Delta_i^L(\lfloor h \rfloor_{C_i^L}).$$

We must show that  $\lfloor h \rfloor_{C_i^L} = \lfloor f \rfloor_{C_i^L}$  for all  $1 \leq i \leq n$ .

Any element  $f \in \bigvee_{i=1}^N C_i^L$  is a descendant of or equal to an element in each  $C_i^L$ . In other words, for every  $i \in \{1, \dots, n\}$ , there exists some  $c_i \in C_i^L$  such that  $c_i \leq f$ . Because  $c_i$  is a prefix of  $f$ , it is exactly the truncation of  $f$  under  $C_i^L$ , so  $c_i = \lfloor f \rfloor_{C_i^L}$ .

By hypothesis,  $h \geq f$ . Since the prefix relation is transitive,  $c_i \leq f$  and  $f \leq h$  imply  $c_i \leq h$ . Because  $C_i^L$  is a prefix code,  $c_i$  is the unique element in  $C_i^L$  that is a prefix of  $h$ . Therefore,  $\lfloor h \rfloor_{C_i^L} = c_i = \lfloor f \rfloor_{C_i^L}$ . Substituting this into the product yields the result.  $\square$

**Definition 2.22** (Unnormalized Posterior Function). *The unnormalized posterior function is a mapping from at most properly terminated strings to non-negative real numbers given by*

$$Z_{n,m}(x) = \sum_{\substack{h \in H \\ h \geq x}} L_n(h) P_m(h)$$

**Proposition 2.2.** *Without any likelihood updates, the unnormalized posterior of the root node is 1:*

$$\begin{aligned} Z_{0,m}(') &= \sum_{\substack{h \in H \\ h \geq '}} L_0(h) P_m(h) \\ &= \sum_{h \in H} L_0(h) P_m(h) \\ &= \sum_{h \in H} 1 \cdot P_m(h) && \text{(by definition 2.16)} \\ &= 1 && \text{(by definition 2.14)} \end{aligned}$$

**Lemma 2.2** (Posterior Equals Sum of Children). *if  $x$  is an unterminated string, then*

$$Z_{n,m}(x) = \sum_{a \in \mathcal{A}} Z_{n,m}(x + a)$$

*Proof.* By definition,

$$Z_{n,m}(x) = \sum_{\substack{h \in H \\ h \geq x}} L_n(h) P_m(h).$$

Since  $x$  is an unterminated string, it does not contain the stop character. Because all strings in  $H$  are properly terminated and therefore end with the stop character,  $x \notin H$ . Thus, any properly terminated string  $h \in H$  that has  $x$  as a prefix must be strictly longer than  $x$  ( $|h| > |x|$ ).

Therefore, the character in  $h$  immediately following the prefix  $x$  must be some character  $a \in \mathcal{A}$ . We can partition the set of strings  $\{h \in H \mid h \geq x\}$  into mutually disjoint subsets based on this next character  $a$ :

$$\{h \in H \mid h \geq x\} = \bigcup_{a \in \mathcal{A}} \{h \in H \mid h \geq x + a\}.$$

Because these subsets are disjoint, we can split the summation over them:

$$\begin{aligned} Z_{n,m}(x) &= \sum_{a \in \mathcal{A}} \sum_{\substack{h \in H \\ h \geq x+a}} L_n(h) P_m(h) \\ &= \sum_{a \in \mathcal{A}} Z_{n,m}(x + a). \end{aligned}$$

$\square$

We may now introduce the datastructure at the core of Dotter, the Bayesian Trie. The Bayesian Trie maintains a calculation of the unnormalized posterior probability while allowing for the lazy application of likelihood and prior updates.

**Definition 2.23** (Bayesian Trie Node). A Bayesian trie node  $x$  associated with a string  $s \in S$  is a data structure containing the tuple  $(n, m, Z)$ , accessed via fields:

- $x.n \in \mathbb{N}$ , the “likelihood time” of the node.
- $x.m \in \mathbb{N}$ , the “prior time” of the node.
- $x.Z \in \mathbb{R}^+$ , a computed quantity which should be equal to the unnormalized posterior of the node evaluated at those times, i.e.,  $x.Z = Z_{x.n, x.m}(s)$ .

**Definition 2.24** (Valid Node). A node,  $x$ , is called valid at time  $(N, M)$  if it satisfies one of the following two conditions:

1. **Constant Likelihood:**  $x$  lies beyond the cumulative likelihood update frontier, i.e.,  $x \geq \bigvee_{i=1}^N C_i^L$
2. **No Reweighting:** The likelihood and the prior have changed by a constant factor since the node’s time, i.e.,  $x \geq \bigvee_{i=x.n+1}^N C_i^L$  and  $x \geq \bigvee_{i=x.m+1}^M C_i^P$

**Lemma 2.3** (Current Nodes are Valid). A node is always valid at its own time. I.e.,  $(x.n, x.m) = (N, M)$  implies that  $x$  is valid at time  $(N, M)$ .

*Proof.* Such a node satisfies the “No Reweighting” condition of validity. Because  $(x.n, x.m) = (N, M)$ , the two joins are joins over empty sets and thus equal to the root node. Hence,  $x \geq \bigvee_{i=x.n+1}^N C_i^L$  and  $x \geq \bigvee_{i=x.m+1}^M C_i^P$ .  $\square$

**Definition 2.25** (Bayesian Trie). A Bayesian Trie is a trie of Bayesian trie nodes along with a global tuple describing the current time,  $(N, M)$ .

**Definition 2.26** (Valid Bayesian Trie). A Bayesian Trie is called valid, if all of its nodes are valid for the trie’s global current time,  $(N, M)$ .

**Definition 2.27** (Node Correctness). A node is “correct” if it correctly calculates the unnormalized posterior,

$$x.Z = Z_{x.n, x.m}(x)$$

**Definition 2.28** (Bayesian Trie Correctness). The Bayesian trie is “correct” if all of its nodes are correct.

We may now give the algorithm to advance a node’s time. This algorithm recalculates the unnormalized posterior of a valid node,  $x$ , to a new time,  $(n', m')$ . Recall that we assume oracular access to the branch prior function,  $P_m^{br}(x)$ .

---

**Algorithm 1** Advance Node Time

---

**Require:**  $x$ , the node to update.

**Require:**  $(n, m)$ , the node’s current time.

**Require:**  $(n', m')$ , the new time.

**Require:**  $(N, M)$ , the trie’s current time.

**Require:**  $x$  is valid at time  $(N, M)$ .

- 1: **if**  $x \geq \bigvee_{i=x.n+1}^N C_i^L$  and  $x \geq \bigvee_{i=x.m+1}^M C_i^P$  **then**
  - 2:      $x.Z \leftarrow x.Z \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P})$
  - 3: **else**  $x \geq \bigvee_{i=1}^N C_i^L$
  - 4:      $x.Z \leftarrow P_{m'}^{br}(x) \prod_{i=1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$
  - 5:  $x.n \leftarrow n'$
  - 6:  $x.m \leftarrow m'$
- 

We also provide for advancing the trie to a new global time.

---

**Algorithm 2** Advance Trie Time

---

**Require:**  $(N, M)$ , the trie's current time.

**Require:** *trie* is valid at time  $(N, M)$ .

**Require:**  $(N', M')$ , the new time.

- 1:  $\text{trie}.N \leftarrow N'$
  - 2:  $\text{trie}.M \leftarrow M'$
  - 3:  $F \leftarrow \bigvee_{i=1}^{N'} C_i^L \wedge \left( \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P \right) \quad \triangleright x \geq F \text{ implies } x \text{ is valid at time } (N', M').$
  - 4: Run **AdvanceNodeTime** to update the time of each node in this frontier to  $(N', M')$ .
  - 5: **for** each node  $x$  strictly within  $F$ ,  $x < F$ , proceeding in reverse topological order **do**
  - 6:      $x.Z \leftarrow \sum_{y \in x.\text{children}} y.Z$
  - 7:      $x.n \leftarrow N'$
  - 8:      $x.m \leftarrow M'$
- 

We now show that both algorithms preserve the validity of the trie and its correctness.

**Theorem 2.1** (Advance Node Time Validity). *The **AdvanceNodeTime** algorithm preserves the validity of the node.*

*Proof.* The “constant likelihood” depends only on the global time, so updating the node's time does not affect if this condition is satisfied. The “no reweighting” condition is weakened by increasing the node's time. I.e.,

$$x \geq \bigvee_{i=n+1}^N C_i^L \implies x \geq \bigvee_{i=n'+1}^N C_i^L$$

and,

$$x \geq \bigvee_{i=m+1}^M C_i^P \implies x \geq \bigvee_{i=m'+1}^M C_i^P$$

if  $n' \geq n$ . □

**Theorem 2.2** (Advance Node Time Correctness). *The **AdvanceNodeTime** algorithm preserves the correctness of the node.*

*Proof.* We show this for the two cases of valid nodes.

**Constant Likelihood Case:** If  $x$  satisfies,  $x \geq \bigvee_{i=1}^N C_i^L$ , then,

$$\begin{aligned} Z_{n',m'}(x) &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) L_{n'}(h) \\ &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) \prod_{i=1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}) \\ &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) \prod_{i=1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) && \text{by lemma 2.1} \\ &= P_{m'}^{br}(x) \prod_{i=1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) && \text{by definition 2.15} \end{aligned}$$

which is accomplished by line 4.

**No Reweighting Case:** If  $x$  satisfies,  $x \geq \bigvee_{i=x.n+1}^N C_i^L$  and  $x \geq \bigvee_{i=x.m+1}^M C_i^P$ , then,

$$\begin{aligned}
Z_{n',m'}(x) &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) L_{n'}(h) \\
&= \sum_{\substack{h \in H \\ h \geq x}} \left( L_n(h) \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}) \right) \left( P_m(h) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor h \rfloor_{C_i^P}) \right) \\
&= \sum_{\substack{h \in H \\ h \geq x}} L_n(h) P_m(h) \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor h \rfloor_{C_i^P}) \\
&= Z_{n,m}(x) \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P})
\end{aligned}$$

which is accomplished by line 6.  $\square$

**Lemma 2.4** (Frontier Calculation). *If  $F = \bigvee_{i=1}^{N'} C_i^L \wedge \left( \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P \right)$ , then  $x \geq F$  and  $x$  is valid at time  $(N, M)$  imply  $x$  is valid at time  $(N', M')$ .*

*Proof.* If  $x \geq F$ , then either,  $x \geq \bigvee_{i=1}^{N'} C_i^L$  or  $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$ . The first case is exactly the same as the ‘‘Constant Likelihood’’ branch in the definition of validity at time  $(N', M')$ . So we only need to consider the second case where  $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$ .

The other assumption, that  $x$  is valid at time  $(N, M)$ , also has two cases: either,  $x \geq \bigvee_{i=1}^N C_i^L$  or  $x \geq \bigvee_{i=x.n+1}^N C_i^L \vee \bigvee_{i=x.m+1}^M C_i^P$ .

**Case 1:** If  $x \geq \bigvee_{i=1}^N C_i^L$  and  $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$ , then we may combine the two joins over the likelihood frontiers to write,  $x \geq \bigvee_{i=1}^{N'} C_i^L$ , which is the ‘‘Constant Likelihood’’ branch for validity at time  $(N', M')$ .

**Case 2:** If  $x \geq \bigvee_{i=x.n+1}^N C_i^L \vee \bigvee_{i=x.m+1}^M C_i^P$  and  $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$ , then we may combine the joins over the likelihood and prior frontiers to write,  $x \geq \bigvee_{i=x.n+1}^{N'} C_i^L \vee \bigvee_{i=x.m+1}^{M'} C_i^P$ , which is the ‘‘No Reweighting’’ branch for validity at time  $(N', M')$ .  $\square$

**Theorem 2.3** (Advance Trie Time Validity). *We must show that after running **AdvanceTrieTime**, the trie is valid at its new time,  $(N', M')$ .*

*Proof.* By lemma 2.4, if  $x \geq F$ , then  $x$  is valid at the new time. Otherwise,  $x$  is strictly within  $F$ . All such nodes are updated to have the new time  $(x.n, x.m) = (N', M')$ . By lemma 2.3, these nodes are valid at their new time.  $\square$

**Theorem 2.4** (Advance Trie Time Correctness). *The **AdvanceTrieTime** algorithm preserves the correctness of the trie.*

*Proof.* We show this by induction on the trie.

We are concerned with nodes,  $x \leq F$ , since nodes which are beyond the update boundary,  $x > F$ , are unaffected by the algorithm.

At the start of the algorithm, all nodes are valid and correct.

**Base Case:** If  $x \in F$ , the  $x$  is valid and correct and we have already shown that running **AdvanceNodeTime** on  $x$  preserves its validity and correctness.

**Induction Step:** If  $x < F$ , then, because we process nodes in reverse topological order, by the induction hypothesis, all its immediate children are correct and have time  $(N', M')$ . Thus line 6 recalculates the unnormalized posterior of  $x$  correctly by lemma 2.2.  $\square$

## 2.2 Likelihood Tracking

The preceding algorithm is correct, but we wish to avoid keeping all the  $\Delta_i^L$  in memory globally and the cost of calculating truncations,  $\lfloor x \rfloor_{C_i^L}$ . To do so we introduce a field called “unpushed likelihood” which will be used to track the product of likelihood deltas. Furthermore, we limit ourselves to handling a single likelihood update at a time.

Formally, we define augmented nodes with an  $x.l$  field, describing the unpushed likelihood, and an  $x.n^{lt}$  field, denoting the local “likelihood tracking time” of the node. We define an augmented trie corresponding to these nodes, and augmented with an  $N^{lt}$  field, denoting the global “likelihood tracking time” of the trie.

(Formally, these fields are independent of the global and local likelihood times of the used in the preceding section, although when we put the algorithms together, they will be the same.)

We define our invariant, “push correctness”:

**Definition 2.29** (Push Correctness). *We say that a Bayesian trie  $B$  is push correct for time  $N^{lt}$  if, every node has local likelihood tracking time at most  $N^{lt}$ , and, for every node whose descendant likelihoods have changed by a constant factor, that factor is equal to the product of the “unpushed likelihood” fields of all of its ancestors (including itself). I.e.,*

$$\forall x, x.n^{lt} \leq N^{lt}$$

and

$$\forall x, x \geq \bigvee_{i=x.n^{lt}+1}^{N^{lt}} C_i^L \implies \prod_{y \leq x} y.l = \prod_{i=x.n^{lt}+1}^{N^{lt}} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$$

This is invariant under two very simple algorithms:

---

### Algorithm 3 Push Likelihood

---

**Require:** Bayesian Trie  $B$ , Node  $x$

**Require:**  $B$  is push correct for time  $B.N^{lt}$ .

**Require:**  $\nexists a < x, a.l \neq 1$

$\triangleright x$  does not have ancestors with unpushed likelihood

1:  $y.l \leftarrow y.l * x.l \quad \forall y \in x.children$

2:  $x.l \leftarrow 1$

3:  $x.n^{lt} \leftarrow B.N^{lt}$

---



---

### Algorithm 4 Apply Likelihood Delta

---

**Require:** Bayesian Trie  $B$ , Node  $x$

**Require:**  $B$  is push correct for time  $B.N^{lt}$ .

1: **for** each node  $x \in C_{B.N^{lt}+1}^L$  **do**

2:      $x.l \leftarrow x.l * \Delta_{B.N^{lt}+1}^L(x)$

3:  $B.N^{lt} \leftarrow B.N^{lt} + 1$

---

**Theorem 2.5** (Push Likelihood Correctness). *Algorithm 3 preserves the push correctness of the trie.*

*Proof.* Let  $N = B.N^{lt}$ , and let  $B'$  be the trie after the algorithm is applied. We show that the defining equality of push correctness still holds for every node  $z$ .

First consider  $z \neq x$ . If  $z \not\geq x$ , then none of the ancestors of  $z$  are modified by the algorithm, so  $\prod_{y \leq z} y.l$  is unchanged. If  $z \geq x$ , then the algorithm removes the factor  $x.l$  from  $x$  and multiplies it into each child of  $x$ . Along the ancestor chain of  $z$ , exactly one child of  $x$  occurs, so the product  $\prod_{y \leq z} y.l$  is again unchanged. In either subcase,  $z.n^{lt}$  is unchanged and  $B'.N^{lt} = N$ , so the right-hand side of Definition 2.29 is unchanged as well. Since  $B$  was push correct before the algorithm, the equality continues to hold for  $z$  in  $B'$ .

Now consider  $z = x$ . After the algorithm,  $x.l = 1$  and  $x.n^{lt} = N$ . By the precondition, every strict ancestor  $a < x$  satisfies  $a.l = 1$ . Therefore

$$\prod_{y \leq x} y.l = 1.$$

On the other hand, because  $x.n^{lt} = B'.N^{lt} = N$ , the product on the right-hand side of Definition 2.29 is an empty product, hence also equal to 1. Thus the equality holds for  $x$  as well.

Therefore  $B'$  is push correct for time  $N$ .  $\square$

**Theorem 2.6** (Apply Likelihood Delta Correctness). *Algorithm 4 preserves the push correctness of the trie.*

*Proof.* Let  $N$  be the value of  $B.N^{lt}$  before the algorithm, and let  $B'$  be the trie after the algorithm. Then  $B'.N^{lt} = N + 1$ .

Fix a node  $z$  such that

$$z \geq \bigvee_{i=z.n^{lt}+1}^{N+1} C_i^L.$$

We must show that

$$\prod_{y \leq z} y.l = \prod_{i=z.n^{lt}+1}^{N+1} \Delta_i^L(\lfloor z \rfloor_{C_i^L})$$

holds in  $B'$ .

Since  $z \geq \bigvee_{i=z.n^{lt}+1}^{N+1} C_i^L$ , in particular  $z \geq C_{N+1}^L$ . Hence  $\lfloor z \rfloor_{C_{N+1}^L}$  is defined and is an ancestor of  $z$ . Because  $C_{N+1}^L$  is a prefix code, this is the unique node of  $C_{N+1}^L$  on the ancestor chain of  $z$ .

The algorithm multiplies  $u.l$  by  $\Delta_{N+1}^L(u)$  for each  $u \in C_{N+1}^L$ . Therefore, along the ancestor chain of  $z$ , exactly one factor is introduced, namely

$$\Delta_{N+1}^L(\lfloor z \rfloor_{C_{N+1}^L}).$$

So in  $B'$  we have

$$\prod_{y \leq z} y.l = \left( \prod_{y \leq z} y.l \right)_B \Delta_{N+1}^L(\lfloor z \rfloor_{C_{N+1}^L}).$$

Since  $B$  was push correct for time  $N$ ,

$$\left( \prod_{y \leq z} y.l \right)_B = \prod_{i=z.n^{lt}+1}^N \Delta_i^L(\lfloor z \rfloor_{C_i^L}).$$

Substituting gives

$$\prod_{y \leq z} y.l = \left( \prod_{i=z.n^{lt}+1}^N \Delta_i^L(\lfloor z \rfloor_{C_i^L}) \right) \Delta_{N+1}^L(\lfloor z \rfloor_{C_{N+1}^L}) = \prod_{i=z.n^{lt}+1}^{N+1} \Delta_i^L(\lfloor z \rfloor_{C_i^L}).$$

This is exactly the required equality, so  $B'$  is push correct for time  $N + 1$ .  $\square$

## 2.3 Character Priors from Token Models

*Note:* It is helpful to make three kinds of distinctions: (1) between sets and their elements, (2) between sequences and their elements, and (3) between characters and tokens. We will choose to use uppercase letters for sets, Greek letters for individual characters, and bold font for token sequences. For example,  $\mathbf{A}$  would represent a set of token sequences while  $\alpha$  would represent a character.

Since Dotter uses a letter-by-letter trie, but language models return token probabilities, we face the task of converting a token model to a literal model.

A given literal prefix corresponds to several token paths. For example, the prefix `sci` could be part of the words with the following tokenizations (using the GPT-2 tokenizer):

- `scientific` (single token),
- `science` (single token),
- `sci-ences` (two tokens),

- `sc-iss-or` (three tokens),

We can see that a literal prefix can correspond to multiple token prefixes (empty string, `sci`, and `sc`), and that for a given token prefix, the literal prefix can be generated from multiple terminal tokens (`science`, `scientific`).

Here we introduce a formalization of tokenization and a key property of byte-pair encoders.

**Definition 2.30** (Token). *A token is a non-empty string.*

**Definition 2.31** (Token Set). *A token set is a set of tokens. We will use the notation  $\mathbf{\Lambda}$  to represent our token set.*

**Definition 2.32** (Stop Token). *We will require that our token set contain a token corresponding to the “stop” character,  $\blacksquare$ .*

**Definition 2.33** (Properly Terminated Token Sequence). *A token sequence is properly terminated if and only if it ends with the stop token, with all preceding tokens not being the stop token. I.e.,*

$$\forall i \quad (i = |\mathbf{a}| - 1 \iff \mathbf{a}[i] = \blacksquare)$$

Analogously to the string case, we will use the notation  $\mathbf{H}$  to represent the set of all properly terminated token sequences.

**Definition 2.34** (At Most Properly Terminated Token Sequence). *A token sequence is at most properly terminated if and only if it is either properly terminated or contains no stop token.*

**Definition 2.35** (Token Sequence). *A token sequence is a sequence of tokens.*

**Definition 2.36** (Canonical Token Sequence). *A token sequence  $\mathbf{a}$  is canonical if and only if re-tokenizing its literalization yields the same token sequence:*

$$T(T^{-1}(\mathbf{a})) = \mathbf{a}.$$

We will use the symbol  $\mathbf{C}$  to represent the set of all canonical token sequences.

**Definition 2.37** (Admissible Token Sequence). *A token sequence is admissible if and only if it is canonical and at most properly terminated.*

We will use the symbol  $\mathbf{A}$  to represent the set of all admissible token sequences.

**Definition 2.38** (Literalization Function). *The literalization function,  $T^{-1}$ , maps a token sequence to a string, by concatenating its tokens. I.e.,  $T^{-1}(\mathbf{a}) = \text{Concat}_{i < |\mathbf{a}|}(\mathbf{a}[i])$ .*

**Definition 2.39** (Tokenization Function). *A tokenization function is a mapping from strings to token sequences,  $T : \mathcal{A}^* \rightarrow \mathbf{\Lambda}^*$ , that is inverted by literalization, i.e.,*

$$T^{-1}(T(x)) = x \quad \forall x \in \mathcal{A}^*$$

(but the converse will not usually hold, i.e., different token sequences may correspond to the same string.) We also require that the tokenization function maps properly terminated strings to properly terminated token sequences,

$$T : \mathbf{H} \rightarrow \mathbf{H}$$

so in particular the final stop character of a properly terminated string tokenizes as the stop token.

**Proposition 2.3.** *For every string  $x$ , the token sequence  $T(x)$  is canonical.*

*Proof.* By definition,

$$T(T^{-1}(T(x))) = T(x).$$

□



**Definition 2.40** (String versus Token Sequence Comparison). *Since literalization is a well defined function, we compare token sequences and strings, by first literalizing the token sequence to a string. We write this as  $\mathbf{a} \leq x$ , to mean,  $T^{-1}(\mathbf{a}) \leq x$ . It does not mean that  $\mathbf{a} \leq T(x)$ .*

**Definition 2.41** (Token Sequence versus Complete Prefix Code Comparison). *We also compare token sequences and complete prefix codes by first literalizing the token sequence to a string. We write  $\mathbf{a} \leq C$  to mean  $T^{-1}(\mathbf{a}) \leq C$ , and  $\mathbf{a} > C$  to mean  $T^{-1}(\mathbf{a}) > C$ .*

**Definition 2.42** (Token Sequence versus Token Sequence Comparison). *We compare token sequences by first literalizing them to strings. We write  $\mathbf{a} \leq \mathbf{b}$  to mean  $T^{-1}(\mathbf{a}) \leq T^{-1}(\mathbf{b})$ .*

**Definition 2.43** (Boundary Sufficiency). *The boundary sufficiency property of a tokenization function states that every contiguous subsequence of a canonical token sequence remains canonical when retokenized in isolation. Formally, a tokenization function  $T$  has boundary sufficiency if for every canonical token sequence  $\mathbf{a} \in \mathbf{C}$  and every  $0 \leq i \leq j \leq |\mathbf{a}|$ ,*

$$T(T^{-1}(\mathbf{a}[i:j])) = \mathbf{a}[i:j].$$

Henceforth, we will always assume that our tokenization function has boundary sufficiency. This is because all byte-pair encoders have this property.

**Proposition 2.4** (Token-sequence prefixes of a tokenization are canonical). *If  $\mathbf{b} \leq T(x)$ , then  $\mathbf{b} \in \mathbf{C}$ .*

*Proof.* Since  $T(x) \in \mathbf{C}$  and  $\mathbf{b} \leq T(x)$ , there exists  $j \leq |T(x)|$  such that  $\mathbf{b} = T(x)[:j]$ . Applying boundary sufficiency to the canonical token sequence  $T(x)$  with indices 0 and  $j$  gives

$$T(T^{-1}(\mathbf{b})) = T(T^{-1}(T(x)[:j])) = T(x)[:j] = \mathbf{b},$$

so  $\mathbf{b} \in \mathbf{C}$ . □

We can now formalize a token-based language model.

**Definition 2.44** (Conditional Token Prior Function). *Our conditional token prior function tells us the conditional probability of a token given a token sequence. It is a mapping*

$$\mathbf{Q} : \mathbf{\Lambda}^* \times \mathbf{\Lambda} \rightarrow \mathbb{R}^+$$

*written as  $\mathbf{Q}(\mathbf{t} \mid \mathbf{a})$ , where,*

$$\forall \mathbf{a} \in \mathbf{\Lambda}^*, \sum_{\mathbf{t} \in \mathbf{\Lambda}} \mathbf{Q}(\mathbf{t} \mid \mathbf{a}) = 1.$$

*We also require canonical support:*

$$\mathbf{Q}(\mathbf{t} \mid \mathbf{a}) = 0 \quad \text{whenever } \mathbf{a} \in \mathbf{A} \text{ and } \mathbf{a} + \mathbf{t} \notin \mathbf{A}.$$

*We also require almost-sure termination: for every admissible token sequence  $\mathbf{a} \in \mathbf{A} \setminus \mathbf{H}$ ,*

$$\sum_{\mathbf{s} \in \mathbf{H}} \prod_{i < |\mathbf{s}|} \mathbf{Q}(\mathbf{s}[i] \mid \mathbf{a} + \mathbf{s}[:i]) = 1.$$

*Explanatory Note:* The almost-sure termination condition is an assumption on the conditional token prior, not a lemma. Local normalization of  $\mathbf{Q}(\mathbf{t} \mid \mathbf{a})$  only guarantees that the next token is sampled from a probability distribution; by itself it does not imply that the process eventually emits the stop token with probability 1.

**Definition 2.45** (Conditional Token Prior Function Sequence). *A conditional token prior function sequence is a sequence of conditional token prior functions,  $\{\mathbf{Q}_n\}$ .*

**Definition 2.46** (Token Prior Function). *A token prior function is a mapping from properly terminated token sequences to non-negative real numbers,*

$$\mathbf{P}_m : \mathbf{H} \rightarrow \mathbb{R}^+,$$

*satisfying*

$$\sum_{\mathbf{h} \in \mathbf{H}} \mathbf{P}_m(\mathbf{h}) = 1.$$

**Definition 2.47** (Token Branch Prior Function). *The token branch prior of a token sequence  $\mathbf{a}$  is*

$$\mathbf{P}_m^{br}(\mathbf{a}) := \sum_{\substack{\mathbf{h} \in \mathbf{H} \\ \mathbf{h} \geq \mathbf{a}}} \mathbf{P}_m(\mathbf{h}).$$

**Definition 2.48** (Induced Token Prior). *Given a conditional token prior function  $\mathbf{Q}_m$ , we define the induced token prior on properly terminated token sequences by*

$$\mathbf{P}_m(\mathbf{h}) := \prod_{i < |\mathbf{h}|} \mathbf{Q}_m(\mathbf{h}[i] \mid \mathbf{h}[:i]).$$

**Proposition 2.5** (Induced Token Prior Is a Token Prior). *The induced token prior is a token prior function.*

*Proof.* By definition,  $\mathbf{P}_m : \mathbf{H} \rightarrow \mathbb{R}^+$ . It remains to show that

$$\sum_{\mathbf{h} \in \mathbf{H}} \mathbf{P}_m(\mathbf{h}) = 1.$$

Since the empty token sequence is admissible and not properly terminated, the almost-sure termination assumption gives

$$\sum_{\mathbf{h} \in \mathbf{H}} \prod_{i < |\mathbf{h}|} \mathbf{Q}_m(\mathbf{h}[i] \mid \mathbf{h}[:i]) = 1.$$

By definition 2.48, the summand is exactly  $\mathbf{P}_m(\mathbf{h})$ , so the claim follows.  $\square$

**Lemma 2.5** (Token Branch Prior Product Formula). *For every admissible token sequence  $\mathbf{a} \in \mathbf{A}$ ,*

$$\mathbf{P}_m^{br}(\mathbf{a}) = \prod_{i < |\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]).$$

*Proof.* We split into two cases.

If  $\mathbf{a} \in \mathbf{H}$ , then  $\mathbf{a}$  is already properly terminated. No properly terminated token sequence can be a strict extension of  $\mathbf{a}$ , since that would place the stop token before the final index. Therefore  $\mathbf{a}$  is the unique properly terminated descendant of itself, so

$$\mathbf{P}_m^{br}(\mathbf{a}) = \mathbf{P}_m(\mathbf{a}).$$

By definition 2.48, this is exactly

$$\prod_{i < |\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]).$$

Now suppose  $\mathbf{a} \in \mathbf{A} \setminus \mathbf{H}$ . Every properly terminated token sequence  $\mathbf{h} \geq \mathbf{a}$  can be written uniquely as

$\mathbf{h} = \mathbf{a} + \mathbf{s}$  where  $\mathbf{s} \in \mathbf{H}$ . Therefore,

$$\begin{aligned}
\mathbf{P}_m^{br}(\mathbf{a}) &= \sum_{\substack{\mathbf{h} \in \mathbf{H} \\ \mathbf{h} \geq \mathbf{a}}} \mathbf{P}_m(\mathbf{h}) \\
&= \sum_{\mathbf{s} \in \mathbf{H}} \mathbf{P}_m(\mathbf{a} + \mathbf{s}) \\
&= \sum_{\mathbf{s} \in \mathbf{H}} \left( \prod_{i < |\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]) \right) \left( \prod_{i < |\mathbf{s}|} \mathbf{Q}_m(\mathbf{s}[i] \mid \mathbf{a} + \mathbf{s}[:i]) \right) \\
&= \left( \prod_{i < |\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]) \right) \sum_{\mathbf{s} \in \mathbf{H}} \prod_{i < |\mathbf{s}|} \mathbf{Q}_m(\mathbf{s}[i] \mid \mathbf{a} + \mathbf{s}[:i]) \\
&= \prod_{i < |\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]),
\end{aligned}$$

where the final step is the almost-sure termination assumption in definition 2.44.  $\square$

Henceforth we will study prior function sequences that are derived from token priors induced from conditional token priors by the relation,

$$P_n(h) = \mathbf{P}_n(T(h)).$$

We can now state our first result

**Theorem 2.7** (Token Prior Path Summation). *The branch prior of a string  $a$ , is the sum of the token branch priors of all token sequences that pass through  $a$ .*

$$P^{br}(a) = \begin{cases} 1, & a = \epsilon, \\ \sum_{\mathbf{b} \in \mathbf{A}, \mathbf{b} < a} \mathbf{P}^{br}(\mathbf{b}) \sum_{\mathbf{t} \in \mathbf{A}, a \leq \mathbf{b} + \mathbf{t}} \mathbf{Q}(\mathbf{t} \mid \mathbf{b}), & a \neq \epsilon. \end{cases}$$

*Proof.* We prove the two cases separately.

If  $a = \epsilon$ , then by Definition 2.15,

$$P^{br}(\epsilon) = \sum_{\substack{h \in H \\ h \geq \epsilon}} P(h) = \sum_{h \in H} P(h) = 1.$$

Now let  $a \neq \epsilon$ . Again from Definition 2.15,

$$P^{br}(a) = \sum_{\substack{h \in H \\ h \geq a}} P(h).$$

For each  $h \geq a$ , write  $\mathbf{c} := T(h)$ . Since  $\mathbf{c} \geq a$ , there exists a unique smallest  $j$  such that  $a \leq \mathbf{c}[:j+1]$ . Define  $\mathbf{b} := \mathbf{c}[:j]$  and  $\mathbf{t} := \mathbf{c}[j]$ . By minimality of  $j$ , we have

$$\mathbf{b} < a \leq \mathbf{b} + \mathbf{t}.$$

So each  $h \geq a$  determines a unique pair  $(\mathbf{b}, \mathbf{t})$  with those inequalities.

Grouping the sum over  $h$  by these pairs gives

$$P^{br}(a) = \sum_{\substack{\mathbf{b} \in \mathbf{A} \\ \mathbf{b} < a}} \sum_{\substack{\mathbf{t} \in \mathbf{A} \\ a \leq \mathbf{b} + \mathbf{t}}} \sum_{\substack{h \in H: \\ (\mathbf{b} + \mathbf{t}) \leq T(h)}} P(h).$$

By the defining relation  $P_m(h) = \mathbf{P}_m(T(h))$ , the innermost sum is

$$\sum_{\substack{\mathbf{h} \in \mathbf{H} \\ \mathbf{h} \geq \mathbf{b} + \mathbf{t}}} \mathbf{P}_m(\mathbf{h}),$$

which is exactly  $\mathbf{P}^{br}(\mathbf{b} + \mathbf{t})$  by definition 2.47. By lemma 2.5,

$$\mathbf{P}^{br}(\mathbf{b} + \mathbf{t}) = \mathbf{P}^{br}(\mathbf{b}) \mathbf{Q}(\mathbf{t} \mid \mathbf{b}).$$

Substituting yields

$$P^{br}(a) = \sum_{\mathbf{b} \in \mathbf{A}, \mathbf{b} < a} \mathbf{P}^{br}(\mathbf{b}) \sum_{\mathbf{t} \in \mathbf{A}, a \leq \mathbf{b} + \mathbf{t}} \mathbf{Q}(\mathbf{t} \mid \mathbf{b}),$$

as claimed.  $\square$

## 2.4 Prior Tracking

We now replace the oracular access to the branch prior function with an explicit dynamic program over token contexts. The first observation is that the historical product of prior deltas appearing in line 2 of algorithm 1 is itself a branch-prior ratio.

**Lemma 2.6** (Prior Delta Factorization). *Let  $m \leq m'$  and let  $x$  be an at most properly terminated string satisfying  $x \geq \bigvee_{i=m+1}^{m'} C_i^P$ . Then*

$$P_{m'}^{br}(x) = P_m^{br}(x) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}).$$

*Proof.* By the definition of a prior delta digest sequence,

$$P_{m'}(h) = P_m(h) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor h \rfloor_{C_i^P}) \quad \forall h \in H.$$

Since  $x \geq \bigvee_{i=m+1}^{m'} C_i^P$ , the same argument as in lemma 2.1 shows that for every  $h \geq x$  and every  $i \in \{m+1, \dots, m'\}$ ,

$$\lfloor h \rfloor_{C_i^P} = \lfloor x \rfloor_{C_i^P}.$$

Hence, for all  $h \geq x$ ,

$$P_{m'}(h) = P_m(h) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}).$$

Summing over all properly terminated descendants of  $x$  gives

$$\begin{aligned} P_{m'}^{br}(x) &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) \\ &= \sum_{\substack{h \in H \\ h \geq x}} P_m(h) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) \\ &= \left( \sum_{\substack{h \in H \\ h \geq x}} P_m(h) \right) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) \\ &= P_m^{br}(x) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}), \end{aligned}$$

as claimed.  $\square$

**Corollary 2.1** (Prior Delta Quotient). *Under the hypotheses of lemma 2.6, if  $P_m^{br}(x) > 0$ , then*

$$\prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) = \frac{P_{m'}^{br}(x)}{P_m^{br}(x)}.$$

Thus it suffices to compute  $P_{M'}^{br}(x)$  on the valid frontier. We now formalize a finite dynamic program that does exactly this.

**Definition 2.49** (Token Fan). *Let  $m$  be a prior time, let  $\mathbf{b} \in \mathbf{A}$  be an admissible token sequence, and let  $x$  be an at most properly terminated string. The token fan of  $x$  from context  $\mathbf{b}$  at time  $m$  is*

$$\Phi_m(\mathbf{b}, x) := \sum_{\mathbf{t} \in \mathbf{A}, x \leq \mathbf{b} + \mathbf{t}} \mathbf{Q}_m(\mathbf{t} \mid \mathbf{b}).$$

**Definition 2.50** (Token Ancestor). *For a non-empty string  $x$ , we define its token ancestor by*

$$\tau(x) := T^{-1}(T(x)[-1]).$$

**Definition 2.51** (String-Prefix Context). *For a string  $x$  and a character index  $i \leq |x|$ , we define the string-prefix context*

$$\kappa_x(i) := T(x[:i]).$$

(and for convenience, we allow this notation for negative indices too).

*Explanatory Note:* The sum for a branch prior runs over *string* ancestors of  $x$ , not merely over token ancestors of  $x$  in its own tokenization. For each strict string prefix  $x[:i]$ , we re-tokenize that shorter string to obtain the context  $\kappa_x(i)$ . In general,  $\kappa_x(i)$  need not be a prefix of  $T(x)$ . This is exactly why algorithm 6 must sum over all strict string ancestors.

**Lemma 2.7** (Token Prefix Retokenization). *For every string  $x$  and every  $j \leq |T(x)|$ ,*

$$T(T^{-1}(T(x)[:j])) = T(x)[:j].$$

*Proof.* Since  $T(x) \in \mathbf{C}$ , the claim follows immediately from boundary sufficiency applied to the canonical token sequence  $T(x)$  with indices 0 and  $j$ .  $\square$

**Lemma 2.8** (Nonadmissible Token Branch Priors Vanish). *If a token sequence  $\mathbf{a}$  is not admissible, then*

$$\mathbf{P}_m^{br}(\mathbf{a}) = 0.$$

*Proof.* If  $\mathbf{a}$  is not at most properly terminated, then no properly terminated token sequence can extend  $\mathbf{a}$ , since that would preserve an internal stop token. Hence the index set in definition 2.47 is empty, so  $\mathbf{P}_m^{br}(\mathbf{a}) = 0$ .

Otherwise,  $\mathbf{a}$  is at most properly terminated but not canonical. Let  $j$  be the least index such that  $\mathbf{a}[:j+1]$  is not canonical. Then  $\mathbf{a}[:j]$  is canonical, and because  $\mathbf{a}$  is at most properly terminated, the sequence  $\mathbf{a}[:j+1]$  is not admissible. By admissible support,

$$\mathbf{Q}_m(\mathbf{a}[j] \mid \mathbf{a}[:j]) = 0.$$

Every properly terminated descendant  $\mathbf{h} \geq \mathbf{a}$  has the same prefix  $\mathbf{a}[:j+1]$ , so its induced token prior contains the same zero factor. Therefore  $\mathbf{P}_m(\mathbf{h}) = 0$  for every such descendant, and hence

$$\mathbf{P}_m^{br}(\mathbf{a}) = 0.$$

$\square$

**Theorem 2.8** (String-Ancestor Reindexing). *For every non-empty at most properly terminated string  $x$ ,*

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(\kappa_x(i)) \Phi_m(\kappa_x(i), x).$$

*Proof.* By theorem 2.7,

$$P_m^{br}(x) = \sum_{\mathbf{b} \in \mathbf{A}, \mathbf{b} < x} \mathbf{P}_m^{br}(\mathbf{b}) \Phi_m(\mathbf{b}, x).$$

We now reindex this sum by the literal prefix

$$s := T^{-1}(\mathbf{b}).$$

Since  $\mathbf{b} \in \mathbf{A}$ , in particular  $\mathbf{b}$  is canonical, so

$$\mathbf{b} = T(T^{-1}(\mathbf{b})) = T(s),$$

so  $\mathbf{b} = \kappa_x(|s|)$ . Since  $s < x$ , this index satisfies  $|s| < |x|$ .

Thus the admissible contexts are in one-to-one correspondence with the strict string prefixes  $x[:i]$  for  $0 \leq i < |x|$ , via  $\mathbf{b} = \kappa_x(i) = T(x[:i])$ . Substituting this reindexing into the path-summation formula yields

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(\kappa_x(i)) \Phi_m(\kappa_x(i), x),$$

as claimed.  $\square$

We provide two short algorithms to compute the priors. The essential thing is to know the “token branch prior”, written  $x.\mathbf{P}$ , of the ancestors of a node before computing its branch prior,  $x.P$ . We augment nodes with times tracking when these calculations are valid,  $x.m^P$  and  $x.m^{\mathbf{P}}$ .

Unlike our likelihood tracking, and our core algorithm, which both lazily reuse calculations across iterations, this section requires us to rerun the prior calculations from the root on each update. This makes the presentation simple.

---

**Algorithm 5** Compute Token Branch Prior

---

**Require:** Bayesian Trie  $B$ , Node  $x$ , New Time  $m$

**Require:** Every node in  $B$  stores correct prior values for its recorded times.

**Require:**  $x = \epsilon$  or  $\tau(x).m^{\mathbf{P}} = m$   $\triangleright$  The token ancestor of  $x$  already has the new time.

- 1: **if**  $x = \epsilon$  **then**
  - 2:      $x.\mathbf{P} \leftarrow 1$
  - 3: **else**
  - 4:      $x.\mathbf{P} \leftarrow \tau(x).\mathbf{P} * \mathbf{Q}_m(T(x)[-1] \mid T(x)[: -1])$
  - 5:  $x.m^{\mathbf{P}} \leftarrow m$
- 

---

**Algorithm 6** Compute Branch Prior

---

**Require:** Bayesian Trie  $B$ , Node  $x$ , New Time  $m$

**Require:** Every node in  $B$  stores correct prior values for its recorded times.

**Require:**  $(x[:i]).m^{\mathbf{P}} = m \quad \forall i < |x|$   $\triangleright$  All string ancestors of  $x$  already have calculated the token branch prior at the new time.

- 1:  $x.P \leftarrow \sum_{i=0}^{|x|-1} (x[:i]).\mathbf{P} * \Phi_m(\kappa_x(i), x)$   $\triangleright$  Theorem 2.8.
  - 2:  $x.m^P \leftarrow m$
- 

We now show the correctness of these two algorithms.

**Theorem 2.9** (Compute Token Branch Prior Correctness). *Algorithm 5 correctly computes the token branch prior of a node.*

*Proof.* We must show that after the algorithm runs,

$$x.\mathbf{P} = \mathbf{P}_m^{br}(T(x)).$$

There are two cases.

If  $x = \epsilon$ , the algorithm sets  $x.\mathbf{P} \leftarrow 1$ . Also  $T(\epsilon) = \epsilon$ , so it suffices to show that

$$\mathbf{P}_m^{br}(\epsilon) = 1.$$

By definition 2.47,

$$\mathbf{P}_m^{br}(\epsilon) = \sum_{\mathbf{h} \in \mathbf{H}} \mathbf{P}_m(\mathbf{h}),$$

which is equal to 1 because  $\mathbf{P}_m$  is a token prior function. Hence

$$\mathbf{P}_m^{br}(T(x)) = \mathbf{P}_m^{br}(\epsilon) = 1 = x.\mathbf{P},$$

so the stored value is correct.

Now suppose  $x \neq \epsilon$ . By the precondition, the token ancestor  $\tau(x)$  already has time  $m$ , so its stored value is correct:

$$\tau(x).\mathbf{P} = \mathbf{P}_m^{br}(T(\tau(x))).$$

By lemma 2.7,

$$T(\tau(x)) = T(x)[-1].$$

Thus,

$$\tau(x).\mathbf{P} = \mathbf{P}_m^{br}(T(x)[-1]).$$

The algorithm then sets

$$x.\mathbf{P} \leftarrow \tau(x).\mathbf{P} \cdot \mathbf{Q}_m(T(x)[-1] \mid T(x)[-1]).$$

Substituting the known ancestor value gives

$$x.\mathbf{P} = \mathbf{P}_m^{br}(T(x)[-1]) \cdot \mathbf{Q}_m(T(x)[-1] \mid T(x)[-1]).$$

By lemma 2.5,

$$\mathbf{P}_m^{br}(T(x)) = \mathbf{P}_m^{br}(T(x)[-1]) \cdot \mathbf{Q}_m(T(x)[-1] \mid T(x)[-1]).$$

Therefore  $x.\mathbf{P} = \mathbf{P}_m^{br}(T(x))$ , as required.

Finally, the assignment  $x.m^{\mathbf{P}} \leftarrow m$  records exactly the time at which this value is valid.  $\square$

**Theorem 2.10** (Compute Branch Prior Correctness). *Algorithm 6 correctly computes the branch prior of a node.*

*Proof.* We must show that after the algorithm runs,

$$x.P = P_m^{br}(x).$$

By the precondition, every strict string ancestor  $x[i]$  has already had its token branch prior computed at time  $m$ . Thus for every  $i < |x|$ ,

$$(x[i]).\mathbf{P} = \mathbf{P}_m^{br}(T(x[i])).$$

By theorem 2.8,

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(\kappa_x(i)) \Phi_m(\kappa_x(i), x).$$

Using definition 2.51, this becomes

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(T(x[i])) \Phi_m(T(x[i]), x).$$

Replacing each  $\mathbf{P}_m^{br}(T(x[i]))$  by the already-computed stored value  $(x[i]).\mathbf{P}$  gives

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} (x[i]).\mathbf{P} \Phi_m(\kappa_x(i), x).$$

This is exactly the value assigned to  $x.P$  by the algorithm. Therefore the algorithm correctly computes the branch prior of  $x$ .

Finally, the assignment  $x.m^{\mathbf{P}} \leftarrow m$  records the time at which this branch prior value is valid.  $\square$

Two implementation remarks are useful in practice:

1. The relevant contexts for a literal prefix  $x$  are governed only by token boundaries that can occur within the final token overlapping  $x$ . When tokens cannot contain spaces internally, this means that only prefixes of the final word need be considered. For a tokenizer with maximum token length  $L$ , this limits the search to at most the last  $L$  characters.
2. If the vocabulary is stored in lexicographic order and the model output is converted to cumulative probabilities, then the fan  $\Phi_M(\mathbf{b}, x)$  can be obtained from a difference of cumulative sums over the contiguous lexicographic range of tokens  $\mathbf{t}$  satisfying  $x \leq T^{-1}(\mathbf{b} + \mathbf{t})$ .

## 2.5 Maximum Descendant Likelihood

Finally, we discuss how to track the maximum token descendant likelihood for nodes. The quantity is not needed to compute posteriors and is orthogonal to the preceding discussion. However, it will be helpful later for determining which token sequences should be evaluated by the language model.

The algorithm somewhat resembles the core posterior calculation algorithm, in that we begin by fixing the computation at the frontier and then fix it within the frontier with a kind of up-propagation.

**Definition 2.52** (Maximum Likelihood for Double Path). *The so called “maximum likelihood for double path” is the maximum likelihood of any descendant,  $h$ , for which the canonical tokenization is an extension of  $\mathbf{a}$  and makes  $\mathbf{a}$  the token truncation of  $x$ . If there is no such descendant, this maximum is taken to be 0.*

$$R_n(\mathbf{a}, x) = \max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} L_n(h)$$

In the special case where the string corresponds to the token sequence, this reduces to,

**Lemma 2.9** (Identical Token and String Prefixes). *If  $\mathbf{a}$  is canonical, i.e.,  $a = T^{-1}(\mathbf{a})$ , then*

$$R_n(\mathbf{a}, a) = \max_{\substack{h \in H \\ \mathbf{a} \leq T(h)}} L_n(h)$$

*Proof.* If  $\mathbf{a}$  is properly terminated then  $h = a$  satisfies the conditions. Otherwise, if  $\mathbf{a}$  is not terminated, then  $h = a + \blacksquare$  satisfies the conditions, since the stop character is always required to tokenize as its own token.  $\square$

This latter quantity, the maximum likelihood of any descendant of a token sequence, is the quantity of interest, but we will need a datastructure that maintains  $R_n(\mathbf{a}, x)$  for other values of  $x$  which are not equal to  $\mathbf{a}$ .

The core theorem required for our algorithm related the double path likelihood to the double path likelihood near an arbitrary frontier.

**Theorem 2.11** (Double Path Maximum Likelihood from the Frontier). *Given a canonical token sequence  $\mathbf{a}$ , a string  $x$ , a complete prefix code  $F$ , and a likelihood time  $n$ , such that  $x \leq F$  and  $H \geq F$ ,*

$$R_n(\mathbf{a}, x) = \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{\mathbf{b} \in C \\ \mathbf{a} \leq \mathbf{b} \leq f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f),$$

*Proof.* We prove both inequalities.

First, let  $h \in H$  be any string satisfying

$$x \leq h, \quad \mathbf{a} \leq T(h), \quad \lfloor x \rfloor_{T(h)} = \mathbf{a}.$$

Because  $H \geq F$  and  $h \in H$ , there exists  $f \in F$  such that  $f \leq h$ . Since also  $x \leq h$  and  $x \leq F$ , we cannot have  $f < x$ , because then  $f$  and the frontier element extending  $x$  would be comparable distinct elements of the prefix code  $F$ . Thus  $x \leq f \leq h$ .



Let

$$\mathbf{b} := \lfloor f \rfloor_{T(h)}.$$

Then  $\mathbf{b} \leq T(h)$ , so proposition 2.4 gives  $\mathbf{b} \in \mathbf{C}$ . Also  $\mathbf{b} \leq f$  and  $\mathbf{a} \leq \mathbf{b}$ . Moreover, by lemma 2.11,

$$\lfloor x \rfloor_{\mathbf{b}} = \lfloor x \rfloor_{\lfloor f \rfloor_{T(h)}} = \lfloor x \rfloor_{T(h)} = \mathbf{a}.$$

Since  $f \leq h$ ,  $\mathbf{b} \leq T(h)$ , and  $\lfloor f \rfloor_{T(h)} = \mathbf{b}$ , the string  $h$  is admissible in the definition of  $R_n(\mathbf{b}, f)$ . Hence

$$L_n(h) \leq R_n(\mathbf{b}, f).$$

Because every admissible  $h$  for  $R_n(\mathbf{a}, x)$  yields such a pair  $(\mathbf{b}, f)$ , this proves

$$R_n(\mathbf{a}, x) \leq \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{\mathbf{b} \in \mathbf{C} \\ \mathbf{a} \leq \mathbf{b} \leq f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f).$$

For the reverse inequality, let  $\mathbf{b} \in \mathbf{C}$  and  $f \in F$  satisfy

$$x \leq f, \quad \mathbf{a} \leq \mathbf{b} \leq f, \quad \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}.$$

If there is no  $h \in H$  admissible in the definition of  $R_n(\mathbf{b}, f)$ , then  $R_n(\mathbf{b}, f) = 0 \leq R_n(\mathbf{a}, x)$  by definition. Otherwise, let  $h \in H$  satisfy

$$f \leq h, \quad \mathbf{b} \leq T(h), \quad \lfloor f \rfloor_{T(h)} = \mathbf{b}.$$

Since  $x \leq f \leq h$ , we have  $x \leq h$ . Also, again by lemma 2.11,

$$\lfloor x \rfloor_{T(h)} = \lfloor x \rfloor_{\lfloor f \rfloor_{T(h)}} = \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}.$$

Because  $\mathbf{a} \leq \mathbf{b} \leq T(h)$ , the string  $h$  is admissible in the definition of  $R_n(\mathbf{a}, x)$ . Hence every value contributing to  $R_n(\mathbf{b}, f)$  also contributes to  $R_n(\mathbf{a}, x)$ , so

$$R_n(\mathbf{b}, f) \leq R_n(\mathbf{a}, x).$$

Taking the maximum over all such  $\mathbf{b}$  and  $f$  gives

$$\max_{\substack{f \in F \\ x \leq f}} \max_{\substack{\mathbf{b} \in \mathbf{C} \\ \mathbf{a} \leq \mathbf{b} \leq f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f) \leq R_n(\mathbf{a}, x).$$

Therefore the two sides are equal. □

**Lemma 2.10** (Canonical token prefixes of a string). *Let  $f$  be a string. The canonical token sequences  $\mathbf{b}$  satisfying  $\mathbf{b} \leq f$  are exactly those of the form*

$$\mathbf{b} = T(f[:i])$$

for some  $i \leq |f|$ .

*Proof.* First suppose  $\mathbf{b} \leq f$  and  $\mathbf{b}$  is canonical. Then there exists a string  $b = T^{-1}(\mathbf{b})$  with  $b \leq f$ . Writing  $i := |b|$ , we have  $b = f[:i]$ , and therefore

$$\mathbf{b} = T(b) = T(f[:i]).$$

Conversely, let  $i \leq |f|$ . Then  $f[:i] \leq f$ , so

$$T^{-1}(T(f[:i])) = f[:i] \leq f.$$

Thus  $T(f[:i])$  is canonical and satisfies  $T(f[:i]) \leq f$ . □

**Corollary 2.2** (Double Path Maximum Likelihood from the Frontier, Reindexed). *Given a canonical token sequence  $\mathbf{a}$ , a string  $x$ , a complete prefix code  $F$ , and a likelihood time  $n$ , such that  $x \leq F$  and  $H \geq F$ ,*

$$R_n(\mathbf{a}, x) = \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{i \leq |f| \\ \lfloor x \rfloor_{T(f[:i])} = \mathbf{a}}} R_n(T(f[:i]), f).$$

*Proof.* By theorem 2.11,

$$R_n(\mathbf{a}, x) = \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{\mathbf{b} \in \mathbf{C} \\ \mathbf{a} \leq \mathbf{b} \leq f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f).$$

For fixed  $f \in F$ , lemma 2.10 shows that the canonical token sequences  $\mathbf{b}$  satisfying  $\mathbf{b} \leq f$  are exactly those of the form  $\mathbf{b} = T(f[:i])$  for  $i \leq |f|$ . Reindexing the outer maximum by this correspondence yields the claim.  $\square$

**Definition 2.53** (Token Truncation). *The token truncation of a string  $x$  under a token sequence  $\mathbf{a}$  is the longest prefix of  $\mathbf{a}$  that is also, as a string, a prefix of  $x$ . It maps a string to a token sequence, and is written as  $\lfloor x \rfloor_{\mathbf{a}}$ . (Context makes it clear when this is intended as opposed to complete prefix code truncation, which uses the same notation, but with a complete prefix code instead of a token sequence in the subscript.)*

$$\lfloor x \rfloor_{\mathbf{a}} = \max_{\substack{\mathbf{b} \leq \mathbf{a} \\ \mathbf{b} \leq x}} \mathbf{b}$$

**Lemma 2.11** (Token truncation is transitive along string prefixes). *If  $x \leq f$ , then for every token sequence  $\mathbf{c}$ ,*

$$\lfloor x \rfloor_{\lfloor f \rfloor_{\mathbf{c}}} = \lfloor x \rfloor_{\mathbf{c}}.$$

*Proof.* By definition,

$$\lfloor x \rfloor_{\lfloor f \rfloor_{\mathbf{c}}} = \max_{\substack{\mathbf{b} \leq \lfloor f \rfloor_{\mathbf{c}} \\ \mathbf{b} \leq x}} \mathbf{b}.$$

Since

$$\lfloor f \rfloor_{\mathbf{c}} = \max_{\substack{\mathbf{d} \leq \mathbf{c} \\ \mathbf{d} \leq f}} \mathbf{d},$$

a token sequence  $\mathbf{b}$  satisfies  $\mathbf{b} \leq \lfloor f \rfloor_{\mathbf{c}}$  if and only if  $\mathbf{b} \leq \mathbf{c}$  and  $\mathbf{b} \leq f$ .

Therefore the indexing set above is exactly

$$\{\mathbf{b} \mid \mathbf{b} \leq \mathbf{c}, \mathbf{b} \leq f, \mathbf{b} \leq x\}.$$

Because  $x \leq f$ , every token sequence whose string is a prefix of  $x$  is automatically a prefix of  $f$ . Hence this set is equal to

$$\{\mathbf{b} \mid \mathbf{b} \leq \mathbf{c}, \mathbf{b} \leq x\}.$$

Taking the maximum over this set gives

$$\lfloor x \rfloor_{\lfloor f \rfloor_{\mathbf{c}}} = \max_{\substack{\mathbf{b} \leq \mathbf{c} \\ \mathbf{b} \leq x}} \mathbf{b} = \lfloor x \rfloor_{\mathbf{c}},$$

as claimed.  $\square$

This prepares us for our first algorithm,

We compute a quantity,  $Q(\mathbf{a}, x)$ . In implementation, this is data attached to the node,  $x$ , indexed by its ancestors,  $\mathbf{a}$ . Our goal is to design an algorithm that correctly computes  $Q(\mathbf{a}, x)$  at time  $N^\phi$ , i.e.,  $Q(\mathbf{a}, x) = R_{N^\phi}(\mathbf{a}, x)$ .

---

**Algorithm 7** MTDL Up Propagation

---

**Require:** Complete Prefix Code,  $F$ **Require:**  $H \geq F$ **Require:** New max likelihood time,  $N^\phi$ **Require:**  $x.n^\phi = N^\phi$   $\triangleright$  All frontier nodes already have the new max likelihood tracking time**Require:**  $f \in F \implies Q(\mathbf{d}, f) = R_{N^\phi}(\mathbf{d}, f)$  for all canonical  $\mathbf{d} \leq f$ .  $\triangleright$  MTDLFA Already Correct at Frontier**Require:**  $x < F \implies Q(\mathbf{d}, x) \leq R_{N^\phi}(\mathbf{d}, x)$  for all canonical  $\mathbf{d} \leq x$ .1: **for** each node  $f \in F$  **do**2:   **for**  $i \leq |f|$  **do**3:      $\mathbf{b} \leftarrow T(f[:i])$   $\triangleright$  Canonical tokenizations of each string ancestor of  $f$ 4:     **for**  $j$  from  $|f|$  down to 0 **do**5:        $Q(\lfloor f[:j] \rfloor_{\mathbf{b}}, f[:j]) \leftarrow_{\max} Q(\mathbf{b}, f)$ 6: **for** each node  $x < F$  **do** $\triangleright$  Update the local max likelihood tracking time of all nodes7:    $x.n^\phi \leftarrow N^\phi$ 

---

**Theorem 2.12** (MTDL Up Propagation Correctness). *Under the hypotheses of algorithm 7, if  $H \geq F$ ,  $\mathbf{a} \leq F$ , and  $x \leq F$ , then*

$$R_{N^\phi}(\mathbf{a}, x) = Q(\mathbf{a}, x).$$

*Proof.* Fix  $\mathbf{a} \leq F$  and  $x \leq F$ . By inspection of algorithm 7, the value  $Q(\mathbf{a}, x)$  at termination is the maximum of its initial value and all values  $Q(T(f[:i]), f)$  such that  $f \in F$ ,  $x \leq f$ , and  $\lfloor x \rfloor_{T(f[:i])} = \mathbf{a}$ . Thus

$$Q(\mathbf{a}, x) = \max \left( Q^{\text{init}}(\mathbf{a}, x), \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{i \leq |f| \\ \lfloor x \rfloor_{T(f[:i])} = \mathbf{a}}} Q(T(f[:i]), f) \right).$$

By the initialization hypothesis,  $Q^{\text{init}}(\mathbf{a}, x) \leq R_{N^\phi}(\mathbf{a}, x)$ . Also, for every  $f \in F$  and  $i \leq |f|$ , the frontier hypothesis gives

$$Q(T(f[:i]), f) = R_{N^\phi}(T(f[:i]), f).$$

Therefore

$$Q(\mathbf{a}, x) = \max \left( Q^{\text{init}}(\mathbf{a}, x), \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{i \leq |f| \\ \lfloor x \rfloor_{T(f[:i])} = \mathbf{a}}} R_{N^\phi}(T(f[:i]), f) \right).$$

By the reindexed corollary above, the second term is exactly  $R_{N^\phi}(\mathbf{a}, x)$ . Hence the maximum above is  $\max(Q^{\text{init}}(\mathbf{a}, x), R_{N^\phi}(\mathbf{a}, x)) = R_{N^\phi}(\mathbf{a}, x)$ .  $\square$ 

The preceding algorithm explains how to update within the frontier to a new likelihood time. We now give a simple update algorithm for beyond the frontier.

---

**Algorithm 8** MTDL Update

---

**Require:** Node  $x$ **Require:** New max likelihood tracking time  $n'$ **Require:**  $n' \geq x.n^\phi$   $\triangleright$  The new likelihood time is not earlier than the node's local time**Require:**  $Q(\mathbf{d}, x) = R_{x.n^\phi}(\mathbf{d}, x)$  for all canonical  $\mathbf{d} \leq x$ .**Require:**  $x \geq \bigvee_{i=x.n^\phi+1}^{n'} C_i^L$ 1:  $Q(\mathbf{d}, x) * = \prod_{i=x.n^\phi+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$  for all canonical  $\mathbf{d} \leq x$ 2:  $x.n^\phi \leftarrow n'$ 

---

**Lemma 2.12** (MTDL Likelihood Factorization). *Let  $n \leq n'$  and let  $x$  be a string satisfying*

$$x \geq \bigvee_{i=n+1}^{n'} C_i^L.$$

Then for every canonical token sequence  $\mathbf{a} \leq x$ ,

$$R_{n'}(\mathbf{a}, x) = R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}).$$

*Proof.* By the definition of a likelihood delta digest sequence, for every  $h \in H$ ,

$$L_{n'}(h) = L_n(h) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}).$$

Since  $x \geq \bigvee_{i=n+1}^{n'} C_i^L$ , the same argument as in lemma 2.1 shows that for every  $h \geq x$  and every  $i \in \{n+1, \dots, n'\}$ ,

$$\lfloor h \rfloor_{C_i^L} = \lfloor x \rfloor_{C_i^L}.$$

Hence, for all  $h \geq x$ ,

$$L_{n'}(h) = L_n(h) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}).$$

If there is no  $h \in H$  admissible in the definition of  $R_n(\mathbf{a}, x)$ , then both  $R_n(\mathbf{a}, x)$  and  $R_{n'}(\mathbf{a}, x)$  are 0 by definition, and the claim follows. Otherwise,

$$\begin{aligned} R_{n'}(\mathbf{a}, x) &= \max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} L_{n'}(h) \\ &= \max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} \left( L_n(h) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \right) \\ &= \left( \max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} L_n(h) \right) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\ &= R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}), \end{aligned}$$

as claimed. □

**Theorem 2.13** (MTDL Update Correctness). *Under the hypotheses of algorithm 8, after execution,*

$$Q(\mathbf{a}, x) = R_{n'}(\mathbf{a}, x)$$

*for all canonical  $\mathbf{a} \leq x$ .*

*Proof.* Let  $n := x.n^\phi$  before execution of the algorithm, and fix a canonical token sequence  $\mathbf{a} \leq x$ . By the precondition,

$$Q(\mathbf{a}, x) = R_n(\mathbf{a}, x).$$

By lemma 2.12,

$$R_{n'}(\mathbf{a}, x) = R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}).$$

The algorithm multiplies  $Q(\mathbf{a}, x)$  by this same factor, so the updated value satisfies

$$\begin{aligned} Q(\mathbf{a}, x) &= R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\ &= R_{n'}(\mathbf{a}, x). \end{aligned}$$

Since  $\mathbf{a}$  was arbitrary, the conclusion holds for all canonical  $\mathbf{a} \leq x$ . □

## 2.6 The Complete Algorithm

Here we combine the preceding sections to give Dotter's complete algorithm. The tracking of the maximum descendant likelihood is optional and will only run on the server.

---

### Algorithm 9 Complete Prior Update

---

**Require:** Bayesian Trie  $B$ .

- 1:  $M' \leftarrow B.M + 1$
  - 2:  $F \leftarrow \emptyset$
  - 3: **for** each node  $x$  in the trie in topological order **do**
  - 4:   Run algorithm 5, Compute Token Branch Prior, on  $x$ .
  - 5:   **if**  $x \geq \bigvee_{i=1}^N C_i^L$  or  $x \geq C_{M'}^P$  **then**
  - 6:      $F \leftarrow F \cup \{x\}$
  - 7:     Break descent from  $x$
  - 8: **for** each node  $x \in F$  **do**
  - 9:   Run algorithm 6, Compute Branch Prior, on  $x$ .
  - 10:   Run algorithm 1, Advance Node Time, on  $x$ .
  - 11: **for** each node  $x < F$  in reverse topological order **do**
  - 12:    $x.Z \leftarrow \sum_{y \in x.\text{children}} y.Z$
  - 13:    $x.m \leftarrow M'$
  - 14:  $B.M \leftarrow M'$
- 

---

### Algorithm 10 Complete Likelihood Update

---

**Require:** Bayesian Trie  $B$ .

- 1:  $N' \leftarrow B.N + 1$
  - 2: **for** each node  $x < C_{N'}^L$  in topological order **do**
  - 3:   Run algorithm 3, Push Likelihood, on  $x$ .
  - 4: **for** each node  $x \in C_{N'}^L$  **do**
  - 5:   Run algorithm 4, Apply Likelihood Delta, on  $x$ .
  - 6:   Run algorithm 1, Advance Node Time, on  $x$ .
  - 7:   Run algorithm 8, MTDL Update, on  $x$ .
  - 8: Run algorithm 7, MTDL Up Propagation, on  $B$  for frontier,  $C_{N'}^L$ .
  - 9: **for** each node  $x < C_{N'}^L$  in reverse topological order **do**
  - 10:    $x.Z \leftarrow \sum_{y \in x.\text{children}} y.Z$
  - 11:    $x.n \leftarrow N'$
  - 12:  $B.N \leftarrow N'$
-

- 1: Run algorithm 5, Compute Token Branch Prior, on  $x$ .
- 2: Run algorithm 6, Compute Branch Prior, on  $x$ .
- 3: Run algorithm 8, MTDL Update, on  $x$ .
- 4: Run algorithm 3, Push Likelihood, on  $x$ .

**Step 1: Select initial frontier:** The “LLM Result” corresponds to a token sequence,  $\mathbf{A}$ . We choose the corresponding character sequence,  $S$ , in the bayesian trie as our initial frontier.

**Step 2: Down Propagation:** We descend from the initial frontier via valid descent until we reach a valid update leafset. A node is known to belong to the valid update leafset, and no further descent from that node is necessary, if and only if the node satisfies one of the following two cases:

- **Valid Likelihood:** The node has never been a parent to a visible node, i.e., it has a valid likelihood.
- **Space Character:** The node is a space character. Formally, the condition is that there exists some token following upon  $\mathbf{A}$  which every possible suffix of our node  $S$ , must contain, i.e., we require that  $S$  satisfy,

$$\exists \mathbf{B}, \mathbf{B} > \mathbf{A}, |\mathbf{B}| = |\mathbf{A}| + 1, \forall Y \geq S, \mathbf{B} < \text{Tokenize}(Y)$$

but in practice we implement this by recognizing that this condition is always true of the space character and rarely true otherwise.

**Note Bene:** Why is descending to the space character necessary? After all, if we know exactly how a node’s prior has changed, and we know that its likelihood has not changed, then we should be able to correctly recalculate the unnormalized posterior, just as how in the likelihood update we could recalculate the unnormalized posterior of a node if we knew how its likelihood had changed and that its prior had remained constant. The subtle fallacy in this thinking is that the *likelihood of the node has changed*. Although we often think of the likelihood as being independent of the prior, this is only true for pure hypotheses. For a composite hypothesis, like this node we have,

$$P(D|S) = \sum_i P(D|S_i)P(S_i|S)$$

where  $S_i$  are the children of  $S$ . Since the weighting, i.e., the prior, has changed, so has the *likelihood* of this node. Thus we do not satisfy case 4 above, because the likelihood has changed.

|                         | <b>Likelihood Update</b>     | <b>Prior Update</b>                      |
|-------------------------|------------------------------|------------------------------------------|
| Mathematical Term       | $P(D   X)$                   | $P(X)$                                   |
| Recompute Posteriors    | Yes                          | Yes                                      |
| Resize Boxes            | Yes                          | Yes                                      |
| How Many New Nodes?     | Dozens                       | Zero to a few                            |
| How to Set Phases       | Optimized / gradient descent | Random (or ancestor / minimum placement) |
| Human-Initiated         | Yes                          | No                                       |
| Show Immediately        | Yes                          | No — wait for ancestor to finish         |
| Branch of Tree Affected | Entire tree                  | Single branch                            |

Table 2.1: Comparison of likelihood updates and prior updates.

## Chapter 3

# Determine Visible Nodes

In this step we identify which prefixes have sufficient posterior probability to be visually displayed.

The two questions are which prefixes to show and whether to show them immediately or to wait until showing them.

### 3.1 Setting the Visibility Threshold

The visibility threshold will determine the number of visible nodes, which entails a trade-off.

**Advantages of a high threshold and few visible nodes:**

- There is less clutter and it is less overwhelming to the user.
- Nodes are larger, avoiding the need for small fonts and timers. The user can recognize timers more easily.
- Faster computation and rendering.

**Advantages of a low threshold and many visible nodes:**

- More granular targets allows us to spread out the probability more evenly, i.e., we can set the timers to make a better approximation to the uniform distribution. (Refer to chapter 5.)

Thus, we want to set the visibility threshold as high as we can while still being able set the timers to approximate a uniform distribution.

### 3.2 Searching for Visible Nodes

In order to identify all nodes exceeding the visibility threshold, we descend the trie. We know that posterior probability decreases monotonically with depth, hence we may stop descending upon reaching a node whose (normalized) posterior probability is less than the visibility threshold. It is necessary to descend the trie in the manner described in chapter 2, since this guarantees that the unnormalized posterior probabilities we observe are valid. (Searching for visible nodes with a flat scan over all nodes would be invalid, since many of their posteriors would be stale, since their updates have been deferred.)

### 3.3 Responding to Likelihood Updates versus Prior Updates

Every time our trie is updated, we need to recalculate the visible nodes. Thus the set of visible nodes will change in response to prior updates as well as in response to likelihood updates. However, these should not be handled in the same way. Since a likelihood update is triggered by the user's gesture, the user expects that the visible nodes will change in response. On the other hand, a prior update is triggered by the background evaluation of a language model. Immediately redrawing in response to these updates would be jarring and disorienting. It could be especially problematic if a child appeared right before the user was about to select



its parent, causing the user to make an error (select the parent instead of the child), through no fault of their own. Similarly, it would be problematic if a node disappeared right as the user was about to select it.

To mitigate this issue, we enforce a couple of rules about redrawing the trie in response to prior updates:

- A new visible node cannot be drawn while its parent is close to culmination.
- A visible node can not be removed until the next likelihood update.

## Chapter 4

# Render Trie

### 4.1 Embellishments

#### 4.1.1 Branches of the Tree

## Chapter 5

# Stimulus Optimization: Ideal Timer Placement

### 5.1 The Continuous One Bit Channel

#### 5.1.1 Two Interpretations

The continuous one bit channel can be conceived in two ways: as a continuous channel with a one bit range, or as a series of times of identical events. The equivalency follows from the fact that the continuous channel may be characterized by the times at which its signal changes. To have a finite (differential) capacity we must constrain the channel to a minimum duration of signal, or equivalently, a minimum time between events. For our purpose, we prefer the latter interpretation of the one bit channel, since the user communicates using a sequence of identical events such as blinks or taps.

#### 5.1.2 The Ideal Timing Distribution

What then, is the capacity of the one bit channel? We prefer the event interpretation for analysis. We consider the ideal distribution of the duration between events. Let  $T$  be the minimum duration between events and  $X$  be the additional delay beyond the minimum, so that the total duration between events is  $T + X$ . Let  $A = T + E[X]$  be the expected inter-arrival time. The rate is given by

$$C = \max_{\mathcal{P}_X} \frac{h(X)}{A}.$$

We refer to the rate-maximizing distribution of  $X$ ,  $\mathcal{P}_X$ , as the ideal timing distribution. We will proceed to show that the ideal timing distribution is an exponential distribution.

We treat the capacity, expected value, and differential entropy as functionals of the probability density function  $f(x)$ . To find the optimal distribution, we introduce a Lagrange multiplier  $\mu$  for the normalization constraint  $\int f(x)dx = 1$  and define the functional  $J(f)$ :

$$J(f) = \frac{h(X)}{A} - \mu \left( \int f(x)dx - 1 \right)$$

We maximize  $J(f)$  by setting the variational derivative with respect to  $f(x)$  to zero:

$$\frac{\delta J}{\delta f(x)} = 0$$

Using the quotient rule for variational derivatives, and noting that  $\frac{\delta h(X)}{\delta f(x)} = -1 - \ln f(x)$  and  $\frac{\delta A}{\delta f(x)} = \frac{\delta E[X]}{\delta f(x)} = x$ , we have:

$$\frac{\delta}{\delta f(x)} \left( \frac{h(X)}{A} \right) = \frac{A(-1 - \ln f(x)) - h(X)x}{A^2}$$

Substituting this into the stationarity condition:

$$\frac{A(-1 - \ln f(x)) - h(X)x}{A^2} - \mu = 0$$

Solving for  $f(x)$ :

$$f(x) \propto \exp\left(-\frac{h(X)}{A}x\right)$$

proving that  $X$  is exponentially distributed with rate parameter equal to the rate of the channel (i.e.,  $\lambda^* = C$ ). This fact lets us solve for the rate,

$$C = \frac{1 - \ln C}{T + 1/C}$$

which by algebra has the solution,

$$C = W(T)/T$$

, where  $W$  is the Lambert  $W$  function.

## A Geometric Intuition

The above result (that the ideal timing distribution is exponential) may be appreciated more readily with a geometric argument. We consider a large sample,  $\mathbf{X}$ , in  $N$ -dimensional space, representing the sequence of additional delays  $X_i$ . By the law of large numbers, its  $L_1$  norm,  $\|\mathbf{X}\|_1 = \sum_i X_i$ , must be tightly distributed about the  $N$ -simplex,  $\|\mathbf{X}\|_1 = NE[X]$ . This can be thought of as the permissible region for  $\mathbf{X}$ . Thus the entropy maximizing distribution for  $\mathbf{X}$  should assign a uniform distribution over the  $N$ -simplex, which is precisely what the exponential distribution does as it depends only on the  $L_1$  norm,  $f(\mathbf{X}) \propto \exp(-\lambda\|\mathbf{X}\|_1)$ .

(Incidentally, this conception also provides a geometric proof of the fact that the differential entropy of the exponential distribution is  $1 - \ln \lambda$ , since the entropy of a uniform distribution over the  $N$ -simplex should be the logarithm of its volume. Since its side length is  $N/\lambda$ , its volume is  $(N/\lambda)^N/N!$ , and its entropy per sample (by Stirling's approximation) is

$$h(X) = \lim_{N \rightarrow \infty} \frac{1}{N} (N(\log(N - \log \lambda) - N(\log N - 1))) = 1 - \ln \lambda$$

as expected.)

## 5.2 Periodic Signals

### 5.2.1 Random Scan Times

### 5.2.2 Periodic signals

### 5.2.3 Approximating A Uniform Distribution with a Gaussian Mixture

UNFINISHED

We have considered the ideal timing distribution and the periodic timing distribution. How can we induce a uniform distribution on the user's signal? There are several targets that the user may aim for, corresponding to all the prefixes displayed. Each target has an assigned signal time,  $\mu_i$  and a fixed and known (mixing) probability,  $\pi_i$ . Given the noise  $\mathcal{E} \sim \mathcal{N}(0, \sigma)$ , the distribution of the observed signal is a mixture of gaussians with means  $\mu_i$  and constant variance  $\sigma^2$ . Given the mixing probabilities,  $\pi_i$ , how should we position the  $\mu_i$  to maximize the entropy of the signal?

Intuitively we want to space apart the  $\mu_i$  to approximate a uniform distribution. Those components with higher  $\pi_i$  should receive more space.

As soon as we receive a signal we recompute the posterior probabilities of the targets and must assign new phases to them. Hence this computation is on the critical path and ought to be computed in a few

milliseconds. This rules out heavier techniques like gradient descent on a numerical approximation of the entropy.

We discuss two heuristic approaches to the problem as well as an approximation for the entropy which may be computed efficiently.

## Two Heuristics

UNFINISHED We wish to space the components of the mixture as to approximate a uniform distribution. One heuristic is to determine (by binary search) a value  $C$  such that

$$\sum_i 2w_i = P$$

where the widths allotted to each component,  $w_i$ , are given by

$$w_i = \text{Re} \left( \sigma \sqrt{2 \left( \ln \left( \frac{\pi_i}{\sqrt{2\pi}\sigma} \right) - C \right)} \right)$$

where  $P$  is the length of the interval. The means are then given by  $\mu_i = w_i + \sum_{j=1}^{i-1} 2w_j$ .

This heuristic is derived from approximating the entropy of the mixture as  $h(X) \approx -\ln f_{\min}$  where  $f_{\min}$  is the minimum value of the mixture density, and approximating the mixture by its greatest component at a given point,

$$f(x) \approx \max_i f_i(x)$$

The value of  $C$  represents the minimum log density of the approximated mixture, i.e., the minimum over the interval of the maximum log density of the components. The width  $2w_i$  is the range over which the  $i$ th component may provide a log density greater than  $C$ . If  $\ln f_i(\mu_i) < C$ , where  $f_i(\mu_i)$  denotes the maximum density of the  $i$ th component at  $x = 0$ , then clearly  $w_i = 0$ , otherwise it is as given above. Our algorithm thus maximizes the minimum density of the maximum component over the interval. Although somewhat crude, this heuristic is efficient.

A refinement may be made by instead approximating the mixture by the sum of its two greatest components, and continuing to try to maximize the minimum density of the distribution. In this case, adjacent components should be spaced apart such that the log of the sum of their densities attains a minimum value of  $C$ . Given two adjacent components,  $i$  and  $i+1$ , we are interested in their separation,  $l_i$ . Letting  $\mu_i = 0$  and  $\mu_{i+1} = l_i$ , we may solve for the minimum value of the two-component mixture:

$$f(x) = \pi_i \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right) + \pi_{i+1} \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left(-\frac{(x-l_i)^2}{2\sigma^2}\right)$$

Note that completing the square for the sum of squared distances gives:

$$(x - \mu_1)^2 + (x - \mu_2)^2 = 2 \left( x - \frac{\mu_1 + \mu_2}{2} \right)^2 + \frac{(\mu_1 - \mu_2)^2}{2}$$

Taking the derivative with respect to  $x$  and setting it to zero we have,

$$\frac{d}{dx} f(x) = \frac{1}{\sqrt{2\pi}\sigma^2} \left( \frac{-2x\pi_i}{2\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right) - \frac{-2(x-l_i)\pi_{i+1}}{2\sigma^2} \exp\left(-\frac{(x-l_i)^2}{2\sigma^2}\right) \right) = 0$$

Algebra yields,

$$\ln \left( \frac{Rx}{l_i - x} \right) = \frac{x l_i}{\sigma^2} - \frac{l_i^2}{2\sigma^2}$$

, where  $R = \pi_i/\pi_{i+1}$ . Taken with the condition that  $\ln f(x) = C$  the two variables,  $C$  and  $R$ , implicitly determine  $l_i$ . So, using a precomputed table of  $l_i$  values for a range of  $C$  and  $R$  values, we can proceed as before, using binary search to find the value of  $C$  such that  $\sum_i l_i = P$ . As before, we will exclude components with  $\ln f_i(\mu_i) < C$  from calculations. As long as each adjacent component has a maximum log density greater than  $C$ , it will be possible to space them such that the log of their sum attains a minimum value of  $C$ .

## Approximations to the entropy based on pairwise distances

In order to optimize the placement of the means to maximize entropy, we may minimize a proxy objective  $J$  based on pairwise distances between the components. We consider approximations to the entropy for both Gaussian and von Mises mixture models.

**Gaussian Mixture Model with Rényi Entropy** The Rényi 2-entropy, also known as collision entropy, is defined as  $H_2(X) = -\ln \int p(x)^2 dx$ . Maximizing this entropy is equivalent to minimizing the information potential  $J = \int p(x)^2 dx$ . For a mixture of Gaussians  $p(x) = \sum_{i=1}^N \pi_i \mathcal{N}(x|\mu_i, \sigma^2)$ , the integral of the squared density is:

$$J = \int \left( \sum_{i=1}^N \pi_i \mathcal{N}(x|\mu_i, \sigma^2) \right)^2 dx = \sum_{i=1}^N \sum_{j=1}^N \pi_i \pi_j \int \mathcal{N}(x|\mu_i, \sigma^2) \mathcal{N}(x|\mu_j, \sigma^2) dx$$

The integral of the product of two Gaussians is itself a Gaussian evaluated at the difference of their means:

$$\int \mathcal{N}(x|\mu_i, \sigma^2) \mathcal{N}(x|\mu_j, \sigma^2) dx = \mathcal{N}(0|\mu_i - \mu_j, 2\sigma^2) = \frac{1}{\sqrt{4\pi\sigma^2}} \exp\left(-\frac{(\mu_i - \mu_j)^2}{4\sigma^2}\right)$$

By dropping the leading scalar and the self-interaction terms ( $i = j$ ) since they are constant with respect to the optimal placement of the means, the objective to minimize simplifies to the pairwise sum:

$$J = \sum_{i < j} \pi_i \pi_j \exp\left(-\frac{(\mu_i - \mu_j)^2}{4\sigma^2}\right)$$

**Gaussian Mixture Model with Kolchinsky Objective** Kolchinsky and Tracey [1] introduced a family of computationally efficient estimators for mixture entropy that rely on calculating pairwise distances between component distributions, proving they form a strict lower bound on the true mixture entropy. For homoskedastic Gaussian mixtures, maximizing this lower bound utilizing the Bhattacharyya distance yields a similar pairwise objective. The effective variance in the penalty term is simply doubled compared to the Rényi objective:

$$J = \sum_{i < j} \pi_i \pi_j \exp\left(-\frac{(\mu_i - \mu_j)^2}{8\sigma^2}\right)$$

**Von Mises Mixture Model with Rényi Entropy** When the components are distributed on the unit circle as von Mises distributions with homoskedastic concentration  $\kappa$ , we again aim to minimize the integral of the squared density. The integral of the cross-term of two von Mises components evaluates to a modified Bessel function of the first kind of order zero,  $I_0$ . Stripping out constants and self-terms, the objective becomes:

$$J = \sum_{i < j} \pi_i \pi_j I_0\left(2\kappa \cos\left(\frac{\mu_i - \mu_j}{2}\right)\right)$$

**Von Mises Mixture Model with Kolchinsky Objective** Applying Kolchinsky's lower bound using the Bhattacharyya coefficient to the von Mises mixture model yields an objective with an identical structure to the Rényi case. The difference lies solely in the concentration parameter, which is halved:

$$J = \sum_{i < j} \pi_i \pi_j I_0\left(\kappa \cos\left(\frac{\mu_i - \mu_j}{2}\right)\right)$$

## An approximation for the entropy via Cramér's theorem

UNFINISHED Here we discuss a closed form approximation for the entropy in terms of the mixing weights,  $\pi_i$ , and the chosen means,  $\mu_i$ . This enables efficient gradient descent on the means  $\mu_i$ . We take a geometric approach and consider a large sample,  $\mathbf{X}$ , in  $N$ -dimensional space. Let  $A_i$  be a discrete random variable

taking on the values of  $\mu_i$ , with respective probabilities  $\pi_i$ . Let  $\mathcal{E}_i \sim \mathcal{N}(0, \sigma^2)$  be random noise on the  $i$ th component. Then we have that

$$\mathbf{X} = \mathbf{A} + \mathcal{E}$$

And

$$\begin{aligned} H(\mathbf{X}) &= H(\mathbf{A}, \mathbf{X}) - H(\mathbf{A}|\mathbf{X}) \\ &= H(\mathbf{A}, \mathcal{E}) - H(\mathbf{A}|\mathbf{X}) \quad \text{If } \mathbf{A} \text{ is fixed, } \mathbf{X} \text{ and } \mathcal{E} \text{ differ only by a translation} \\ &= H(\mathcal{E}) + H(\mathbf{A}) - H(\mathbf{A}|\mathbf{X}) \\ &= H(\mathcal{E}) + I(\mathbf{A}; \mathbf{X}) \end{aligned}$$

Since the entropy of the noise is constant, our objective is to maximize the mutual information between  $\mathbf{X}$  and the component choice,  $\mathbf{A}$ . We frame this in terms of trying to infer the true component choice,  $\mathbf{A}$ , from the observed sample,  $\mathbf{X}$ . To this end, we consider the prior distribution of the vertices' distances to the sample,  $D = \|\mathbf{X} - \mathbf{B}\|_2^2$ , where  $\mathbf{B}$  represents a candidate component choice and is distributed identically to  $\mathbf{A}$ . Noting that,  $D = \sum_i D_i$  where  $D_i = (A_i + \mathcal{E}_i - B_i)^2$ , we have that

$$P(\bar{D} = d) \approx \exp(-NI(d))$$

and the likelihood for  $\bar{D}$  is

$$P(\mathbf{X}|\bar{D} = d) = \exp\left(-\frac{1}{2\sigma^2}Nd\right)$$

Thus the posterior mode of  $\bar{D}$  is attained where  $N(I(d) + \frac{d}{2\sigma^2})$  is minimized, which occurs when  $I'(d) = -\frac{1}{2\sigma^2}$ . At this point, the prior probability of the posterior mode will be  $\exp(-NI(d))$ , meaning that the observation delivers  $NI(d)$  bits of information about the component choice, or  $I(d)$  bits per sample.

$I(d)$  had a closed form expression, although its derivation is a somewhat formidable piece of algebra. We begin by defining the random variable  $C_i = A_i - B_i$ , which is also a discrete random variable taking on the values of  $d_{ij} = \mu_i - \mu_j$  with probabilities  $\rho_{ij} = \pi_i\pi_j$ .

Solving for  $I$  in terms of its derivative, and setting it to  $-\frac{1}{2\sigma^2}$ , we have that

$$\mathbf{I}\left(\theta = -\frac{1}{2\sigma^2}\right) = \frac{1}{2}\ln(\mathbf{Z}) - \frac{1}{4} - \ln(\mathbf{Z}) - \frac{1}{8\sigma^2}\frac{\mathbf{Y}}{\mathbf{Z}}$$

where

$$\mathbf{Z} = \sum_{i,j} \rho_{ij} \exp\left(-\frac{\mathbf{d}_{ij}^2}{4\sigma^2}\right)$$

and

$$\mathbf{Y} = \sum_{i,j} \rho_{ij} \mathbf{d}_{ij}^2 \exp\left(-\frac{\mathbf{d}_{ij}^2}{4\sigma^2}\right)$$

The final term may be recognized as the expected squared distance of  $C_i$  under a Boltzmann distribution.

## 5.3 User Parameters

### 5.3.1 The Likelihood Model

We model our observations of the user's signal as being subject to two sources of noise: an outlier probability,  $\rho$ , that the signal was unintended, and additive noise on intended signals. We model the unintended signals as uniformly distributed, and the noise as a Gaussian. (We do not require that the noise have zero mean, since the user may be consistently late or early, relative to the intended time.) Hence our likelihood model for the received signal is a uniform-gaussian mixture model, characterized by the outlier probability,  $\rho$ , and the parameters of the noise,  $\mu_\epsilon, \sigma_\epsilon$ .

### 5.3.2 Directional Statistics



## Chapter 6

# Render Stimulus

### 6.1 Circular Timers

In this step we represent the nodes' phases to the user. Dotter does this by drawing a small circular “timer” for each node.

We justify this choice with reference to the concept of “local attention”.

One visually simpler choice would be to let a “selection” bar descend the screen from top to bottom, and to adjust nodes' vertical positions to reflect their phases. This approach resembles that row and column selection method of many communication boards.

Although this is less cluttered, it requires the use of global attention. By letting each node carry its own timer, the user does not need to be aware of any part of the screen outside of their foveal vision.

## Chapter 7

# Detect Response

Dotter requires the precise timing of the user's signal. Because the user's signal's variance is often as little as 20 milliseconds, and signal detection frames might be spaced as far apart as 30 milliseconds, the noise introduced by the signal detection process can be on the order of the noise introduced by the user.

### 7.1 Blink Detection

This frame frequency problem arises when the user's signal is a blink. A typical laptop webcam might capture 40 frames per second. If we use a simple threshold to detect a blink, this will introduce on the order of 20 milliseconds of noise. To resolve this problem we should use interpolation. The blink signal can be linearly interpolated between the frame before and the frame after the threshold was crossed to estimate the precise time of the blink. In post-hoc analysis making this change reduced about 3 milliseconds of noise when my blink standard deviation was about 20 milliseconds.

Further noise reductions should be made through a simple linear regression of features of the blink signal against the correct blink times. This can remove another millisecond of noise.

### 7.2 Keypress Detection

For an in-browser implementation, the `keydown` event should be preferred at the time should be read directly from the `event.timeStamp` field which provides the precise time that the keydown event was received by the browser.

## Chapter 8

# Determine Most Likely Break Nodes

One challenge of Dotter is that likelihoods are received with reference to a character-based trie, but priors are received with reference to a token-based trie. A considerable portion of the implementation is devoted to closing that gap.

In the chapter 3 we saw how to identify the character sequence prefixes with the highest posterior probabilities so that we could display them to the user. Here our task is to identify the token sequence prefixes with the highest posterior probabilities, so that we can query the language model for predictions. Just as we solved that problem with a character-based trie, so here we will solve it with a token-based trie. A token-based trie has a very high radix, equal to the model’s vocabulary size, e.g., 50,257 for GPT-2.

### 8.1 An Upper Bound for the Likelihood of a Token Sequence

In the character-based trie (see chapter 2), calculating the likelihoods was trivial, but owing to the token-based nature of the language model, calculating the priors of a character sequence was somewhat involved. Here the situation is reversed. Calculating the prior of a token sequence is trivial, since that is directly what the language model returns, while calculating the likelihood of a token sequence prefix is difficult.

For any given token prefix  $A$ , and any token prefix sequence partition set  $T$  (e.g. the set of all tokens, i.e., all token sequences of length 1), the likelihood of  $A$  is given by,

$$P(D|A) = \sum_{t \in T} P(D|A+t)P(A+t|A)$$

However, this poses a problem since we assume that  $P(A+t|A)$  is not known. We are seeking to evaluate the likelihood of  $A$  in order to decide the priority of querying the language model for its next token, but in order to calculate this likelihood, we must already know the prior probabilities of the next token conditioned on  $A$ ! Thus we are in a catch-22 and must resort to using a heuristic for the likelihood.

Since the likelihood of  $A$  is a weighted sum of its descendants’ likelihoods, it must be bounded by the minimum and maximum of its descendants’ likelihoods.

$$\min_{t \in T} P(D|A+t) \leq P(D|A) \leq \max_{t \in T} P(D|A+t)$$

Since we don’t want to rely upon our current model of the prior, we require choose  $T$  so that all the descendants  $A+t$  have a valid likelihood, i.e., so that they have never been a parent of a visible node.

We will prioritize token prefixes by their upper bound on the likelihood. Thus it is necessary to know, for a given node, the maximum likelihood of any descendant which has this node as a token ancestor. This is referred to as the `max_token_descendant_likelihood` of the node.

Maintaining this value during likelihood updates is discussed in chapter ??.

## 8.2 Descending the Token Trie

Our choice of upper bound has the property that it strictly decreases with depth. This is because if a given node,  $x$ , is a token descendant of  $B$ , and  $B$  has  $A$  as a token ancestor, then  $x$  must be a token descendant of  $A$  as well and thus `A.max_token_descendant_likelihood`  $\geq$  `B.max_token_descendant_likelihood`.

We make a priority-first search of the token trie, visiting the node with the highest upper bound on its posterior probability first. When we visit a node about which we have not yet queried the language model, we query the language model (chapter 9) about this node. The next token probabilities are referred to as the “LLM Result” which can then be consumed to update the character-based trie with a prior update (chapter ??). Our descent of the token trie is not interrupted by the prior update, but we will need to restart our descent from the root after any likelihood update.

When we visit a node we must add its children to the priority queue. However, the radix is very high, equal to the vocabulary size (e.g.,  $V = 50,257$  for GPT-2). Thus we only add the children with the large posterior upper bounds to the queue. In implementation we use  $K = 100$ . To identify these one hundred children with the highest posterior upper bounds, we must calculate the posterior upper bound for all children. The language model itself gives us an array of length  $V$  for the prior probabilities of the children. Our task is to construct the array of likelihood upper bounds. To do so, we descend the character-based trie from the node in the character-based trie which corresponds to our current node in the token-based trie.

We descend the character-based trie until one of two conditions are met:

1. The descendant has a valid likelihood, i.e., it has never been a parent of a visible node. In this case, we know that this likelihood is the likelihood of all possible child token sequences. We set the relevant alphabetical range of the likelihood array to this likelihood. (E.g. if our node is ‘the’, and this descendant with a valid likelihood is ‘the ho’, we must set the likelihood array to this descendant’s likelihood for every token in the range ‘ ho’ to ‘ hp’ such as the tokens ‘ horse’, ‘ home’, ‘ hound’, etc.)
2. It is impossible for deeper descendants to be direct token children of our root node. I.e., the descendant does not extend our node by a strictly partial token prefix. In this case, we are at a direction token sequence child, ‘B’, of our root node, ‘A’. For the specific element of the likelihood array corresponding to the last token of  $B$ , we set the likelihood to the maximum descendant likelihood of  $B$  which could have  $B$  as a token ancestor, i.e., `B.max_token_descendant_likelihood`.

## Chapter 9

# LLM Query

This step is straightforward. We call a model to find the distribution over the next token for a given token sequence. The only challenge is to correctly maintain the model's Key-Value Cache.

### 9.1 Maintaining a Key-Value Cache Trie

Traditionally, when we call a language model to generate a sequence of tokens, our Key-Value Cache is a simple list that grows as the sequence of tokens is extended. In our case, we must use a trie to store the Key-Value Cache.

# Chapter 10

## Mastering Dotter

### 10.1 The Cost of Incorrect Signals

Expertise in Dotter is about accuracy in three ways: (1) The user must synchronize their signals with the target timers. (2) The user must accurately select the correct prefix. (3) The user must not make involuntary signals (e.g. involuntary blinks).

From the program's perspective, there is virtually no difference between the user selecting the wrong prefix and making an involuntary signal. Both are considered as incorrect signals and both are assumed to have random phase. If the user intended the correct timer and was merely imprecise in synchronizing their signal, we do not consider this to be an "incorrect" signal.<sup>1</sup>

Here we will quantify the cost of incorrect signals.

UNFINISHED

The user's speed can be calculated from their performance parameters. Here we consider the effect of  $\rho$ , the probability of unintended signals. Given an observation,  $D$ , the gain in logits for the true hypothesis  $h$  is  $\ln P(D|h) - \ln P(D|\neg h)$ . When the posterior probability of  $h$  is small, this gain is equal to the gain in the log posterior probability of  $h$ . Let  $r$  denote the hypothesis that the signal was unintended, so that  $P(r) = \rho$ .

We state the following facts:

$$\begin{aligned} P(D|\neg h, r) &= P(D|\neg h, \neg r) = P(D|\neg h) \\ P(D|\neg h) &\approx P(D), \quad \text{when } P(h) \approx 0 \\ P(D|h, r) &= P(D), \quad \text{assuming that } D, h \text{ are independent conditioned on } r \end{aligned}$$

The likelihood under the true hypothesis is

$$P(D|h) = \rho P(D|h, r) + (1 - \rho) P(D|h, \neg r)$$

We consider the case where  $P \gg \sigma$  so that the components of the mixture are well-separated. This means that the contribution of the false component is negligible and that we can approximate the likelihood  $P(D|h)$  piecewise as follows:

$$P(D|h) \approx \begin{cases} P(D|h, r)P(r) & \text{if } D \sim P(D|h, r) \\ P(D|h, \neg r)P(\neg r) & \text{if } D \sim P(D|h, \neg r). \end{cases}$$

That is, when  $D$  is likely drawn from the unintended distribution,  $P(D|h) \approx P(D|h, r)P(r)$ . When  $D$  is likely drawn from the intended distribution,  $P(D|h) \approx P(D|h, \neg r)P(\neg r)$ .

---

<sup>1</sup>We may safely assume that an involuntary signal has random phase. As for an incorrectly selected prefix, if we assume (perhaps unreasonably) that the incorrect prefix was randomly selected from all prefixes, and we further assume (legitimately) that we have uniformly distributed the prefixes' timers, then such a signal has random phase as well.

$$\begin{aligned} E_{D|r,h} [\ln P(D|h)] &\approx E_{D|r,h} \left[ \ln \left( P(D|h,r)P(r) \right) \right] \\ &= \ln \rho - h(D|h, r) \end{aligned}$$

and similarly

$$E_{D|\neg r,h} [\ln P(D|h)] \approx \ln(1 - \rho) - h(D|h, \neg r)$$

The expected log likelihood of the data under the true hypothesis is

$$\begin{aligned} E_D [\ln P(D|h)] &= \rho E_{D|r,h} [\ln P(D|h)] + (1 - \rho) E_{D|\neg r,h} [\ln P(D|h)] \\ &\approx \rho(\ln \rho - h(D|h, r)) + (1 - \rho)(\ln(1 - \rho) - h(D|h, \neg r)) \\ &= -H_2(\rho) - \rho h(D|h, r) - (1 - \rho)h(D|h, \neg r) \\ &= -H_2(\rho) - \rho h(D) - (1 - \rho)h(D|h, \neg r), \quad \text{assuming } D|r, h|r, \text{ independent} \end{aligned}$$

Where  $H_2$  is the binary entropy function.

The likelihood of the alternative hypothesis is

$$E_{D|h} [\ln P(D|\neg h)] \approx E_{D|h} [\ln P(D)]$$

To simplify this term we use its expectation over  $h$ . Recalling the fact that,

$$E_a [E_{b|a}[f(b)]] = E_b[f(b)]$$

, we find that this term's expectation over  $h$  is  $-h(D)$ .

So the total expected gain in the logits of our hypothesis is

$$E_{D|h} [\ln P(D|h) - P(D|\neg h)] \approx -H_2(\rho) + (1 - \rho)h(D) - (1 - \rho)h(D|h, \neg r)$$

In our particular model,  $D$  is uniformly distributed over an interval of length  $P$  and the entropy of  $D$  conditional on  $h$  is just the entropy of the gaussian noise. Denoting the latter term by  $H_\sigma$ , we have,

$$E_{D|h} [\ln P(D|h) - P(D|\neg h)] \approx -H_2(\rho) + (1 - \rho)(\ln P - H_\sigma)$$

This has a nice interpretation: we gain  $\ln P - H_\sigma$  bits of information per intended signal, but the presence of unintended signals makes us pay a penalty in two ways: only  $(1 - \rho)$  of the time is our signal intended, and additionally we must expend  $H_2(\rho)$  bits to learn whether the signal was intended.

Unfortunately the singularity of the binary entropy function at  $\rho = 0$  becomes a practical problem for fast text entry. For example, a seemingly low outlier rate of  $\rho = 0.03$  imposes the relatively high cost of  $H_2(0.03) \approx 0.19$  bits in addition to the 3% loss that would be naively expected. Under reasonable parameters for an intermediate user (e.g.  $\sigma = 25ms$ ,  $P = 700ms$ ), these combined terms amount to a loss of more than 10% of the channel's capacity. Thus to achieve expert entry speeds, it is necessary to make unintended signals very rarely.

# Bibliography

- [1] Artemy Kolchinsky and Brendan Tracey. Estimating mixture entropy with pairwise distances. *Entropy*, 19(7):361, July 2017.
- [2] David J. Ward, Alan F. Blackwell, and David J. C. MacKay. Dasher—a data entry interface using continuous gestures and language models. In *Proceedings of the 13th Annual ACM Symposium on User Interface Software and Technology*, UIST '00, pages 129–137, New York, NY, USA, 2000. ACM.